



Design & Innovation Project (DIP)

Project Report

LiDAR and camera fusion for better robot tracking under complex environment

Project Group: E037

**School of Electrical and Electronic Engineering
Academic Year 2024/25
Semester 1**

Table of Contents

1. Purpose and Project Objectives	1
1.1 Introduction	1
1.2 Schedule	3
2. Project Summary	5
2.1 Problem and Motivation	5
2.2 Methodology	5
3. Scope	6
3.1 Center-based 3D Object Detection and Tracking (CenterPoint) using LiDAR	6
3.1.1 Introduction to CenterPoint	6
3.1.2 CenterPoint Detection + Tracking models for NuScenes	6
a. Center Heatmap Head	7
b. Regression Heads	7
c. Velocity Head and Tracking	8
3.1.2.1 Testing CenterPoint NuScene models' results	8
a. Detection model:	8
b. Tracking model:	9
3.1.2.2 Visualization on NuScenes Detection model	10
3.1.3 CenterPoint Detection model for KITTI	10
a Point Cloud Data	11
b Backbone3D	11
c Backbone2D	11
d Densehead	12
e RoIHead	12
3.1.2.1 Testing CenterPoint KITTI Detection	13
3.1.3.2 Visualization on KITTI Detection model	13
3.2 Deep learning-based Computer Vision (CV) using Camera Data	14
3.2.1 Introducing the 2-stage Approach (YOLOv7 and Byte Track Algorithm)	14
3.2.1a Why YOLOv7 as an Object Detector	14
3.2.1b Why YOLOv7 work as a real-time Object Detector	15
3.2.2 ByteTrack Algorithm (Object Tracking)	22

3.2.2a Core of ByteTrack (BYTE data association method)	24
3.2.3 YOLOv7 + ByteTrack Results	25
3.3 Implementation of the YOLOv7 + ByteTrack Algorithm on the Robot	32
How the robot works:	32
3.4 Lidar-CV Fusion	39
3.4.1 Introduction to EagerMOT	39
3.4.2 EagerMOT	39
3.5 Data Collection and Labelling	41
4. Cost	43
5. Outcomes / Benefits	44
6. Project Management Review	45
6.1 Project Initiation	45
6.2 Project Planning	45
6.3 Risk Management	45
6.4 Cost Management	45
7. Reflection	46
8. Acknowledging/Declaring the Use of GAI	47
References	49
Appendix A - Project Members Information	52

1. Purpose and Project Objectives

Project Name:	LiDAR and camera fusion for better robot tracking under complex environment
Project Supervisor:	Associate Professor Lin Zhiping
Members:	<u>LiDAR Sub-group:</u> Nguyen Khanh Nam (Project Manager) Annabella Choo Lim Jing Xuan <u>CV Sub-group:</u> Ravindran Ramalingam (Project Treasurer) Kyaw Tun Khine Nitheesh Sundaram <u>LiDAR-CV Fusion Sub-group:</u> Zhang Yifan (Sub-group Leader) Da Ruolin Mindy Soh Min Qi

Table 1: Summary of Project Name, Supervisor and Members

This project was conducted with the support of the following mentors/collaborators:

- Associate Professor Lin Zhiping
- Song Rongzihan (Research Associate)
- Zhang Xing (EEE Master's Student)
- Wen Jihao (EEE Master's Student)
- Wang Zhiyuan (EEE Master's Student)

1.1 Introduction

As smart cities aim to make use of technology and innovation to improve quality of life, the development of object tracking and localization applications is essential. Light Detection and Ranging (LiDAR) and camera sensors are each widely used and a vital part in these applications. With their own advantages and limitations, the fusion of data from these two sensors might enhance the accuracy of object tracking and reduce error.

Traditional Computer Vision (CV) algorithms could be used to perform Multi-Object Tracking (MOT) and localization. However, With the advent of new Technologies as society progresses, computational power and the amount of data increasingly rapidly, more and more deep learning-based methods are being developed and becoming the norm for object tracking and localization. In addition, deep learning-based computer vision usually outperforms traditional CV in terms of tasks like object detection and

tracking. They are also known to be more flexible and adaptable in complex environments. This is a crucial point as robots typically need to move in changing and complex environments [15].

This project aims to utilize deep learning-based CV and LiDAR techniques which uses data from advanced sensors such as LiDAR and camera sensors to enhance the accuracy of object tracking and localization in robots. To ensure that the main objective can be met, the SMART principles are used in the planning of this project:

- a. Specific: The scope of the project has been narrowed down to focus solely on the fusion of LiDAR and camera sensors in the tracking of robots, thus improving the accuracy of object detection and localization. The focus is to run the algorithms and test if they can produce state-of-the-art results.
- b. Measurable: To quantify the success of the project, the percentage of error reduction in the object tracking is measured and recorded. The goal is to increase the percentage of error reduction in robot tracking (for example, being able to navigate smoothly around more obstacles).
- c. Achievable: This project is achieved by following closely to the schedule drawn up for this project as illustrated in section 4.
- d. Relevant: As both LiDAR and camera sensors each have their own strengths, combining these two might significantly enhance the ability of the robot to accurately navigate through complex environments. Companies like Waymo have been implementing sensor fusion techniques.
- e. Time-bound: The project starts in August and is projected to end in November 2024. The detailed schedule can be found in section 1.2.

1.2 Schedule

Planned and Actual Milestone Timeline:

The table below indicates the planned and actual milestone dates.

Task Milestone	Planned Milestone Date	Actual Milestone Date
Phase 1: Planning		
Learning of basic skills – PyTorch and Python	21 August	21 August
Understanding of project – reading of LiDAR, CV & Fusion Based MOT papers	28 August	28 August
Discussion and completion of project charter	2 September	2 September
Setting up of test environment on servers and test running of GitHub code	11 September	11 September
Phase 2: Execution		
Data collection – collected using two robots (one moving with LiDAR and camera, another static with just a camera)	18 September	18 September
Data collection and data pre- processing into suitable format with bounding boxes for 3D and 2D data	2 October	2 October
Data labelling and Data Formatting into YOLO, KITTI/NuScenes format	9 October	9 October
Model training and hyper parameters tuning for all teams	23 October	23 October
LiDAR: Train model using KITTI dataset and train on our own dataset with new model	23 October	23 October
CV: Model evaluation between FairMOT and YOLOv7 + ByteTrack	30 October	30 October
LiDAR/Fusion: Model Training	30 October	30 October
LiDAR: Visualisation of own data	30 October	30 October

for correct bounding boxes		
CV: Implementation on Robot	6 November	8 November
LiDAR: Conversion of KITTI detection results to NuScenes format for tracking	6 November	6 November
LiDAR/Fusion: Model Evaluation and Visualisation	6 November	6 November
Phase 3: Completion		
Evaluate costs, benefits, outcomes and project management review	11 November	11 November
Final Report Writing	11 November	11 November
Video Making	15 November	15 November
Final Presentation	18 November	18 November

Table 2: Planned and Actual Milestone Dates

For the implementation of robot, the Rospy code took longer to implement than expected because the YOLOv7 + ByteTrack model underperformed, and we had to retrain multiple versions of it.

2. Project Summary

2.1 Problem and Motivation

With the recent advancements in robotics and autonomous vehicles, it has become increasingly important to ensure accuracy and precision in MOT, especially in complex environments (such as in wet weather conditions and crowded places). Currently there has been many approaches on individual sensors such as LiDAR and cameras. There has also been much research on the fusion of the data of these 2 sensors. Each of these approaches has its own advantages and limitations. Camera sensors provide clear images, but their quality deteriorates in complex environments or when objects are partially obstructed from view. On the other hand, LiDAR sensors are beneficial for 3D mapping and can accurately measure distances. However, they are sensitive to changes in environmental conditions such as rain and fog. Hence, this project aims to overcome these limitations by studying individual models with regards to camera and LiDAR data as well as using sensor fusion algorithm to fuse the data of both sensors to tap on their combined advantages and improve the accuracy in object tracking.

2.2 Methodology

To effectively enhance the accuracy of object detection and tracking, we were divided into three specialized groups, each focusing on the implementation of 3 different models.

1. The CV group focuses on training and testing 2 different models: YOLOv7 + ByteTrack and FairMOT, which utilises camera sensor's data from the robot. This group specializes in implementing visual-based algorithms for object identification on the robot. They also plan on integrating one of the 2 models on the experimental Robot.
2. The LiDAR group focuses on the visualization of the trained CenterPoint model which leverages LiDAR sensors to enhance the precision of distance measurement and 3D mapping.
3. The CV + LiDAR group worked independently, using an integrated model to develop a system that leverages both 2D and 3D data from the two sensors for enhanced analysis and implementation

Collaboration between these three groups ensures that the tasks are split evenly and more effectively, hence fulfilling the specific and achievable metrics.

3. Scope

3.1 Center-based 3D Object Detection and Tracking (CenterPoint) using LiDAR

LiDAR was implemented in the algorithms for object-tracking. CenterPoint was utilised. The object-tracking was split into 2 parts: detection and tracking

3.1.1 Introduction to CenterPoint

In the initial stage of our project, accurately detecting objects was essential. To ensure the accuracy of detection, this project made use of a 3D object-detection algorithm known as CenterPoint. CenterPoint is unlike the conventional algorithms which use the bounding-box method to detect objects. Instead, it identifies the centres of each object which improves the rotational invariance and significantly simplifies the detection process. CenterPoint achieved state-of-the-art performance on the nuScenes benchmark. [1]

3.1.2 CenterPoint Detection + Tracking models for NuScenes

Like PointPillars [2] and VoxelNet [3], a standard LiDAR-based backbone network is utilised and is made to build an input point-cloud illustration. This illustration is flattened into an overhead map-view and utilises a standard image-based keypoint detector to identify the centre of the object [4]. For every identified centre, it reverts from the centre location's point-feature to other object properties (3D size, position, and velocity).

For 3D object detection, CenterPoint uses a center-based method where objects are identified based on their center instead of the traditional bounding box method. Figure 1 illustrates the framework of the CenterPoint model. The output of the 3D back-bone is $M \in R^{W \times H \times F}$. The first stage of the CenterPoint makes dense predictions for each object the heatmap of each class, the size, the precise location with a sub-voxel, rotation and velocity.

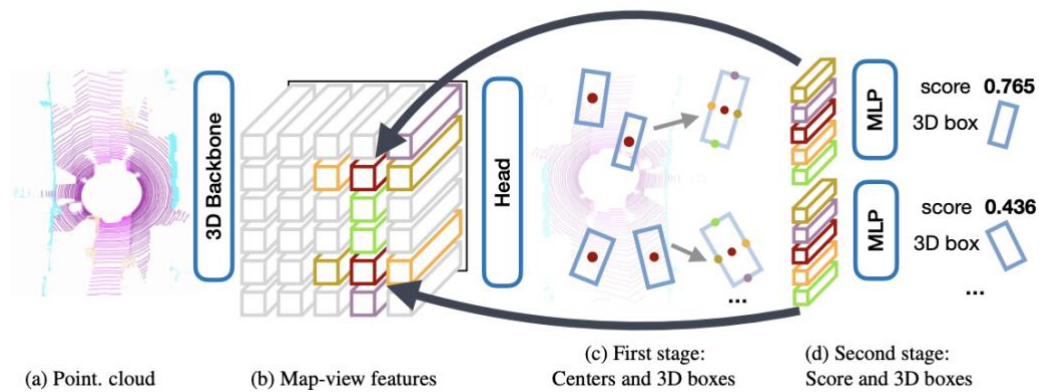


Figure 1: CenterPoint framework.

a. Center Heatmap Head

The goal of the center head is to create a peak on a heatmap at the detected center of each object. The output of a multichannel heatmap is produced, with each channel representing each class. During the training stage, the 3D centers are projected onto a top-down map view to target a 2D Gaussian. In map-view, distances are absolute while an image-view distorts them by perspective. Furthermore, the sparsity of the supervisory signal makes identifying and locating objects difficult. To counteract this, the positive supervision for the target heatmap is done by enlarging the Gaussian peak at each ground truth object's center, specifically by setting the Gaussian radius to $\sigma = \max(f(wl), \tau)$, with $\tau = 2$ being the smallest allowable Gaussian radius. F represents the radius function defined in CornerNet . [6]

b. Regression Heads

Several object properties are stored at point-features of centre objects, such as: sub-voxel location refinement, $o \in \mathbb{R}^2$, height-above-ground, $h_g \in \mathbb{R}$, 3D size, $s \in \mathbb{R}^3$ and yaw rotation angle, $(\sin(\alpha), \cos(\alpha)) \in \mathbb{R}^2$. The sub-voxel location refinement, o addresses the quantization errors that arise from the voxelisation process and the stride operations within the backbone network. The height-above-ground, h_g , component further refines 3D object localization by incorporating missing elevation information lost during map-view projection. The orientation prediction utilizes sine and cosine of the yaw angle as continuous regression targets. Together with box size, the regression heads specify entirely the 3D bounding box. Each output is predicted by its own head. During training, only the ground truth centers are supervised with L1 regression loss. All properties are then extracted, during inference time, by indexing into dense regression head outputs at the peak location of each object. [1]

c. Velocity Head and Tracking

In addition, CenterPoint predicts the 2D velocity $v \in \mathbb{R}^2$ for each detected object, enabling it to track objects over time. To estimate the change in the object's position over time, previous point cloud frames are aligned with the current frame. During interference, CenterPoint uses the estimated closest position and velocity to match the objects in the current frame with past detections. After three frames, unmatched objects are removed. [7]

To achieve precise object localization and classification, the model integrates voxel-based encoding, point based feature extraction, region proposal networks and refinement layers. Each module in the model, Backbone3D, Backbone2D, DenseHead and RolHead, plays a significant role in the feature extraction and prediction process.

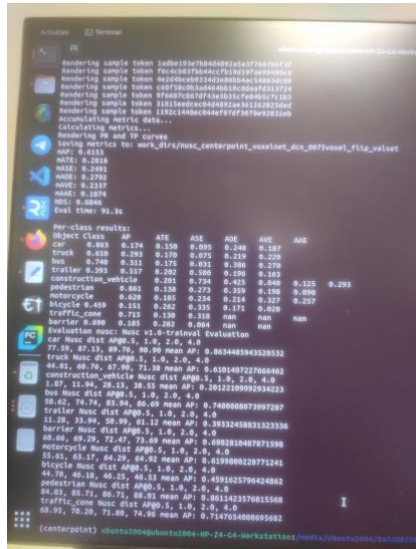
3.1.2.1 Testing CenterPoint NuScene models' results

We trained the detection model using NuScenes dataset and use the generated detection results (a json containing the size of bounding boxes and detection score for each sample token) for tracking model on validation dataset. Our model achieves 61.53 mAP, 68.46 NDS for 3D detection, and 67.0 AMOTA for 3D tracking while the paper results achieve 58.0 mAP, 65.5 NDS for 3D detection, and 63.7 AMOTA for 3D tracking.

1.1.1.1.1 Detection model:

Figure 2a illustrates the performance metrics of the 3D object detection model which evaluates its ability to detect and classify different object classes in point cloud data. The following metrics are used:

- AP (Average Precision): Measures detection accuracy for each object class; higher is better.
- ATE (Average Translation Error): Indicates positional accuracy in meters; lower is better.
- ASE (Average Scale Error): Reflects size estimation accuracy as 1-IOU; lower is better.
- AOE (Average Orientation Error): Orientation estimation error in radians; lower is better.
- AVE (Average Velocity Error): Velocity estimation precision; lower is better.
- AAE (Average Attribute Error): Attribute prediction error as 1-accuracy; lower is better



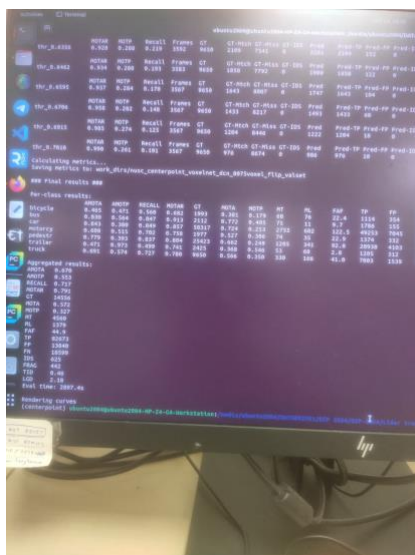
Object Class	AP	ATE (m)	ASE (1-IOU)	AOE (rad)	AVE (m/s)	AAE (1-acc)
car	0.857	0.177	0.144	0.114	0.234	0.121
truck	0.543	0.304	0.177	0.09	0.342	0.145
bus	0.634	0.28	0.16	0.081	0.568	0.346
trailer	0.566	0.378	0.145	0.773	0.213	0.138
construction...	0.215	0.666	0.381	1.086	0.094	0.06
pedestrian	0.84	0.125	0.289	0.351	0.197	0.11
motorcycle	0.591	0.17	0.237	0.257	0.587	0.103
bicycle	0.331	0.194	0.268	0.496	0.239	0.069
traffic_cone	0.822	0.098	0.332	n/a	n/a	n/a
barrier	0.714	0.201	0.246	0.063	n/a	n/a

Figure 2a: Our trained CenterPoint result from NuScenes vs from paper

b. Tracking model:

The following metrics are used:

- AMOTA (Average Multi-Object Tracking Accuracy): Measure how well the tracker maintains accuracy across its lifetime, higher is better
- AMOTP (Average Multi-Object Tracking Precision): Average localization error between predicted and actual object positions, lower is better



Object Class	AMOTA	AMOTP (m)	RECALL	MOTAR	GT	MOTA	MOTP (m)
bicycle	0.331	0.561	0.387	0.72	2186	0.277	0.199
bus	0.715	0.636	0.742	0.834	1701	0.618	0.329
car	0.818	0.391	0.826	0.823	68518	0.676	0.261
motorcycle	0.587	0.519	0.6	0.764	1945	0.456	0.225
pedestrian	0.78	0.409	0.811	0.828	34010	0.662	0.213
trailer	0.693	0.729	0.737	0.807	2566	0.592	0.47
truck	0.625	0.5	0.656	0.714	8639	0.468	0.362

Figure 2b: Our trained CenterPoint Tracking result (for validation set) vs from paper

Comparing both results, the values are similar which indicates that the CenterPoint model works well on our computer environment.

3.1.2.2 Visualization on NuScenes Detection model

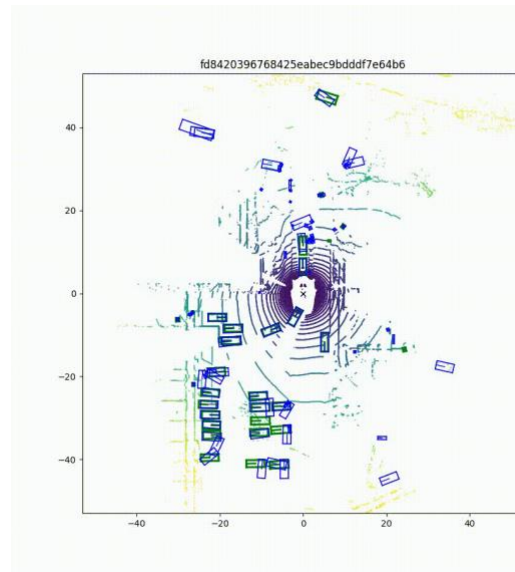


Figure 3: First Frame of Visualisation on Scene 0003 of NuScenes Dataset with correct bounding boxes (full visualisation linked)

3.1.3 CenterPoint Detection model for KITTI

Since our collected data point cloud follow the livox format ((x,y,z, intensity) point cloud inside the lsv file), it is easier to convert into KITTI format, which only have the velodyne folder for the point cloud ((x, y, z, r) point cloud, where (x, y, z) is the 3D coordinates and r is the reflectance value inside bin file). Compared to KITTI, the NuScene format includes 7x more object annotations and the structure is much more complex.

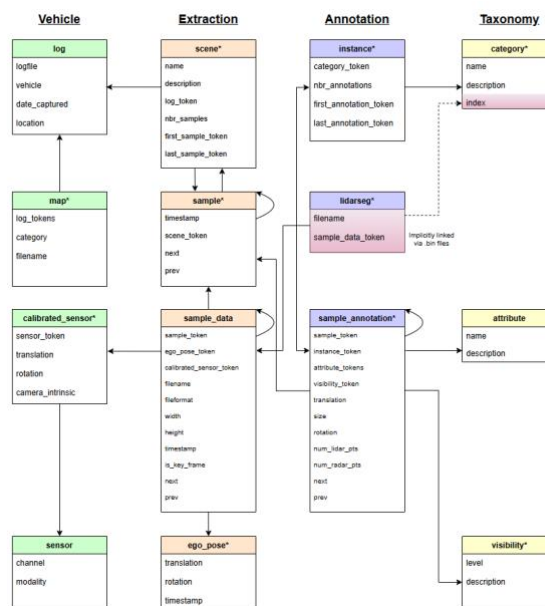


Figure 4: NuScene schema

The CenterPoint-KITTI only provided the detection model, so we need to convert the results into NuScene format for the tracking model

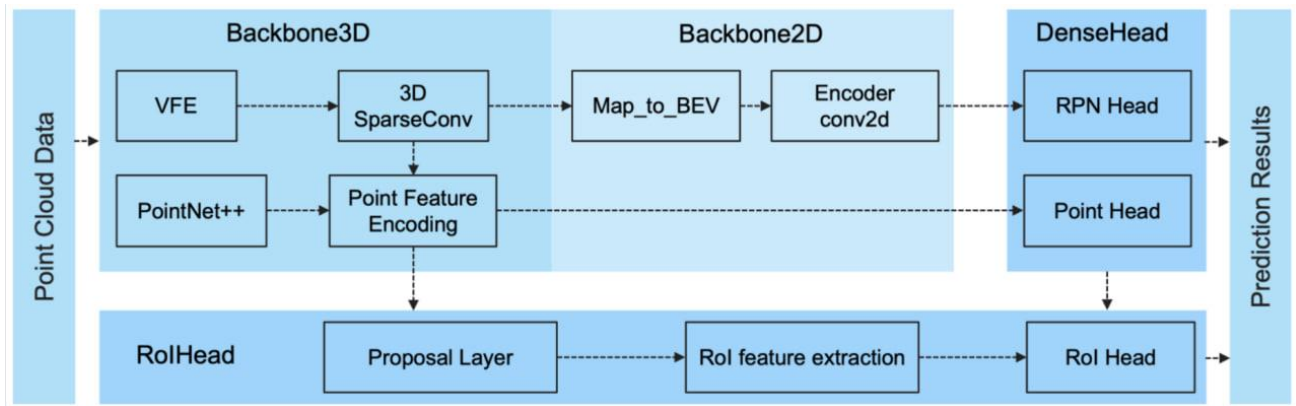


Figure 5: CenterPoint-KITTI Detection Model

a Point Cloud Data

Point clouds represent the 3D coordinates of objects in the environment, which serves as the input for the 3D data detection pipeline.

b Backbone3D

Backbone3D consists of a few components:

- Voxel Feature Encoding (VFE):** This layer transforms the point cloud into a regular 3D grid of voxels for efficient processing in convolutional networks.
- 3D Sparse Convolution (3D SparseConv):** In this layer, sparse 3D convolutions are applied to the voxelated data, enabling spatial information to be captured with reduced computational cost.
- PointNet++:** PointNet++ works simultaneously with the VFE layer by directly processing point cloud data without voxelization, capturing tiny features at multiple layers. This helps to preserve details lost during voxelization.
- Point Feature Encoding:** To create a comprehensive representation of each point in the point cloud, features from VFE and PointNet++.

c Backbone2D

Backbone2D transforms the 3D features into a 2D representation for further processing. The following steps are applicable in this layer:

- Map to Bird's Eye View (BEV):** When the 3D features are transformed into a 2D representation, a BEV is created.

- **Encoder conv2d:** In this step, dimensionality of the feature map is reduced, and critical information is retained for the subsequent steps. This is done by passing the 2D convolutional encoder to the BEV representation to further extract the spatial features from the transformed data.

d Densehead

Based on the encoded features in Backbone2D, the Densehead module generates initial predictions. This module consists of two primary components:

- **Region Proposal Network (RPN) Head:** The RPN head leverages the BEV to identify regions that likely contain objects by creating bounding box proposals.
- **Point Head:** The Point Head refines the characteristics of the objects at point level to enable the model to perform more accurate classifications within each proposal.

e RoIHead

RoIHead refines the predictions generated by Densehead, ensuring the accuracy of the object detection and classification. The RoIHead module contains the following layers:

- **Proposal Layer:** The proposal layer filters and selects regions of interest, prioritizing regions that are more likely to contain objects.
- **Regions of Interest (RoI) Feature Extraction:** This layer extracts features for RoI to refine the predictions generated by the DenseHead. This improves object localization and classification by focusing on specific regions.

3.1.2.1 Testing CenterPoint KITTI Detection

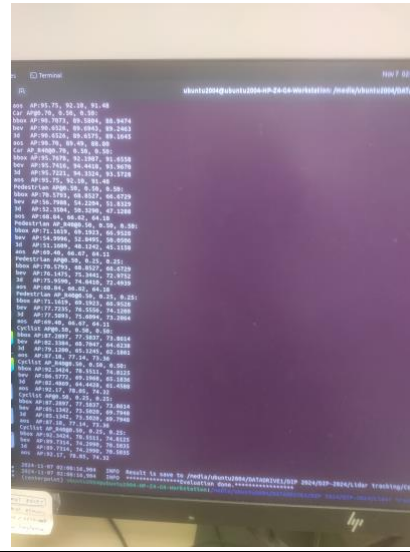


Figure 6: Testing results on KITTI Dataset

3.1.3.2 Visualization on KITTI Detection model

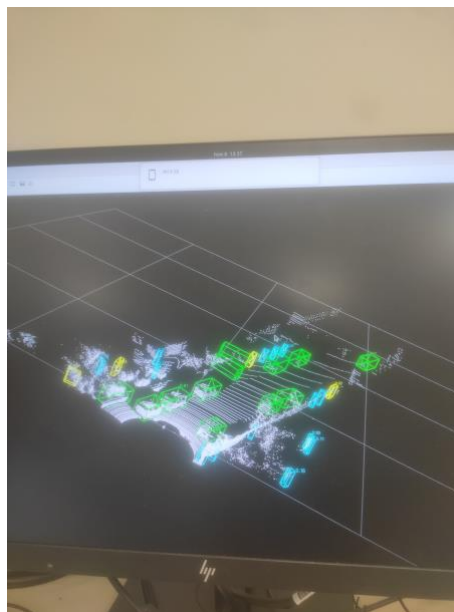


Figure 7: Visualisation on KITTI Dataset Point cloud 000000.bin with correct bounding boxes

3.2 Deep learning-based Computer Vision (CV) using Camera Data

The camera team is responsible for implementing multi-object tracking by utilising camera sensor data to improve object detection and localization.

MOT is a computer vision task [8] aimed at estimating bounding boxes and identities of objects in videos, locating the object across the frames of a video.

Before delving into the algorithm, we should understand 2 key tasks:

- **Object Detection** is to detect the presence of an object in the frame
- **Object Tracking** is to maintain/association of the identities of the object detected across frames.

These are the 2 crucial components to performing multi-object tracking. Given that Deep learning-based Computer Vision Methods are end to end, there is no need for us to handcraft features that capture the information relevant to each task, unlike in traditional computer vision algorithms, as the deep learning models would be able to inherently learn both feature representations and object association directly from the data.

There are multitudes of ways to go about doing multi-object tracking. The difference in these algorithms lie in the fact that either they are 2-stage (object detection first then object tracking) or 1-stage methods (end-to-end detection and tracking).

2-stage methods usually yield higher accuracy in terms of object detection while being slower in terms of speed as compared to 1-stage methods. 1-stage methods can do object tracking better than 2-stage methods [9]. In terms of this project, we are testing both a 2-stage approach (You-Only-Look-Once (YOLOv7)) (detector) + Byte track (Tracker) and 1-stage approach (FairMOT).

The key requirements when considering which algorithm to implement on the robot, for Multi-Object Tracking (MOT) are fast inference and high accuracy when detecting object across frames. We need accurate detection of the object across the frames in real-time.

3.2.1 Introducing the 2-stage Approach (YOLOv7 and Byte Track Algorithm)

In the 2-stage Approach for doing the MOT task, we need an object detector (to detect all objects of interest across all the frames) and an object tracker (does object association whereby it tries to link objects detected in current frame with its corresponding objects from previous frames, also called tracklets).

3.2.1a Why YOLOv7 as an Object Detector

YOLOv7 is a state-of-the-art object detection model. Based on statistics in 2022 [11], in term of speed and accuracy, the YOLOv7 model surpasses all known object detectors in both speed and accuracy in the range from 5 FPS to 160 FPS [10]. From Figure 8 below,

we can infer that the 56.8% AP test-dev / 56.8% AP min-val among all known real-time object detectors with 30 FPS or higher on GPU V100 is the best performing.

Model	#Param.	FLOPs	Size	FPS ^{V100}	AP ^{test} / AP ^{val}	AP ^{test} ₅₀	AP ^{test} ₇₅
YOLOv7-tiny-SiLU	6.2M	13.8G	640	286	38.7% / 38.7%	56.7%	41.7%
PPYOLOE-S [85]	7.9M	17.4G	640	208	43.1% / 42.7%	60.5%	46.6%
YOLOv7	36.9M	104.7G	640	161	51.4% / 51.2%	69.7%	55.9%
YOLOv5-N (r6.1) [23]	1.9M	4.5G	640	159	- / 28.0%	-	-
YOLOv5-S (r6.1) [23]	7.2M	16.5G	640	156	- / 37.4%	-	-
PPYOLOE-M [85]	23.4M	49.9G	640	123	48.9% / 48.6%	66.5%	53.0%
YOLOv5-N6 (r6.1) [23]	3.2M	18.4G	1280	123	- / 36.0%	-	-
YOLOv5-S6 (r6.1) [23]	12.6M	67.2G	1280	122	- / 44.8%	-	-
YOLOv5-M (r6.1) [23]	21.2M	49.0G	640	122	- / 45.4%	-	-
YOLOv7-X	71.3M	189.9G	640	114	53.1% / 52.9%	71.2%	57.8%
YOLOv7-CSP [81]	52.9M	120.4G	640	106	51.1% / 50.8%	69.6%	55.7%
YOLOv7-S [21]	9.0M	26.8G	640	102	40.5% / 40.5%	-	-
YOLOv5-L (r6.1) [23]	46.5M	109.1G	640	99	- / 49.0%	-	-
YOLOv5-M6 (r6.1) [23]	35.7M	200.0G	1280	90	- / 51.3%	-	-
YOLOv7-W6 [81]	96.9M	226.8G	640	87	53.0% / 52.7%	71.4%	57.9%
YOLOv7-W6	70.4M	360.0G	1280	84	54.9% / 54.6%	72.6%	60.1%
YOLOv5-X (r6.1) [23]	86.7M	205.7G	640	83	- / 50.7%	-	-
YOLOv7-M [21]	25.3M	73.8G	640	81	47.2% / 46.9%	-	-
PPYOLOE-L [85]	52.2M	110.1G	640	78	51.4% / 50.9%	68.9%	55.6%
YOLOv7-P6 [81]	37.2M	325.6G	1280	76	53.9% / 53.5%	71.4%	58.9%
YOLOv7-L [21]	54.2M	155.6G	640	69	50.1% / 49.7%	-	-
YOLOv7-W6 [81]	79.8G	453.2G	1280	66	55.2% / 54.8%	72.7%	60.5%
YOLOv5-L6 (r6.1) [23]	76.8M	445.6G	1280	63	- / 53.7%	-	-
YOLOv7-X [21]	99.1M	281.9G	640	58	51.5% / 51.1%	-	-
YOLOv7-E6	97.2M	515.2G	1280	56	56.0% / 55.9%	73.5%	61.2%
YOLOv7-E6 [81]	115.8M	683.2G	1280	45	55.8% / 55.7%	73.4%	61.1%
PPYOLOE-X [85]	98.4M	206.6G	640	45	52.2% / 51.9%	69.9%	56.5%
YOLOv7-D6	154.7M	806.8G	1280	44	56.6% / 56.3%	74.0%	61.8%
YOLOv5-X6 (r6.1) [23]	140.7M	839.2G	1280	38	- / 55.0%	-	-
YOLOv7-E6E	151.7M	843.2G	1280	36	56.8% / 56.8%	74.4%	62.1%
YOLOv7-D6 [81]	151.7M	935.6G	1280	34	56.5% / 56.1%	74.1%	61.9%
F-RCNN-R101-FPN+ [5]	60.0M	246.0G	1333	20	- / 44.0%	-	-
Deformable DETR [100]	40.0M	173.0G	1333	19	- / 46.2%	-	-
Swin-B (C-M-RCNN) [52]	145.0M	982.0G	1333	11.6	- / 51.9%	-	-
DETR DCS-R101 [5]	60.0M	253.0G	1333	10	- / 44.9%	-	-
EfficientDet-D7x [74]	77.0M	410.0G	1536	6.5	55.1% / 54.4%	72.4%	58.4%
Dual-Swin-T (C-M-RCNN) [47]	113.8M	836.0G	1333	6.5	- / 53.6%	-	-
ViT-Adapter-B [7]	122.0M	997.0G	-	4.4	- / 50.8%	-	-
Dual-Swin-B (HTC) [47]	235.0M	-	1600	2.5	58.7% / 58.4%	-	-
Dual-Swin-L (HTC) [47]	453.0M	-	1600	1.5	59.4% / 59.1%	-	-
Model	#Param.	FLOPs	Size	FPS ^{A100}	AP ^{test} / AP ^{val}	AP ^{test} ₅₀	AP ^{test} ₇₅
DN-Deformable-DETR [41]	48.0M	265.0G	1333	23.0	- / 48.6%	-	-
ConvNeXt-B (C-M-RCNN) [53]	-	964.0G	1280	11.5	- / 54.0%	73.1%	58.8%
Swin-B (C-M-RCNN) [52]	-	982.0G	1280	10.7	- / 53.0%	71.8%	57.5%
DINO-Sscale (R50) [89]	47.0M	860.0G	1333	10.0	- / 51.0%	-	-
ConvNeXt-L (C-M-RCNN) [53]	-	1354.0G	1280	10.0	- / 54.8%	73.8%	59.8%
Swin-L (C-M-RCNN) [52]	-	1382.0G	1280	9.2	- / 53.9%	72.4%	58.8%
ConvNeXt-XL (C-M-RCNN) [53]	-	1898.0G	1280	8.6	- / 55.2%	74.2%	59.9%

Figure 8 shows YOLOv7 surpassing all the other state-of-the-art object detectors in 2022[11].

3.2.1b Why YOLOv7 work as a real-time Object Detector

To understand YOLOv7, we would have to first understand the important features of the previous model architectures (from the original YOLO all the way to YOLOv4) that were kept in YOLOv7 before delving into architectural changes which causes YOLOv7 to outperform the previous YOLO versions and other objectors.

In 2015, most state-of-art object detectors were of high accuracy but were much slower as they were 2-stage predictors whereby they first predict the region where the object might be at, called regional proposal, before classifying if it's an object and readjusting the boundaries to better fit the object.[11]

The key idea behind the YOLO families of model is that the image is only passed through the model's network architecture once, hence the name you-only-look-once. It models the way humans look at an image and immediately recognises the object of interest.

In the original YOLO, it introduced dense grid-based predictions. Key Features which were retained by YOLOv7 is unified backbone network (to extract features which are used for prediction), S-by-S grids, bounding box regression, non-max suppression

(NMS), intersection over union (IoU) and the idea of combining 3 type of loss functions (bounding box loss, objectness loss and classification loss) called multi-part loss function. YOLO was built on top a CNN backbone, (backbone network is used to extract key features of the image).

Shown in Figure 9 below, object detection becomes a regression problem. YOLO algorithm first splits the image into S -by- S grids and for each grid it immediately predicts B number of bounding boxes with its coordinates, one confidence score (to determine if there is an object) and C number of classification score for each object in the dataset.

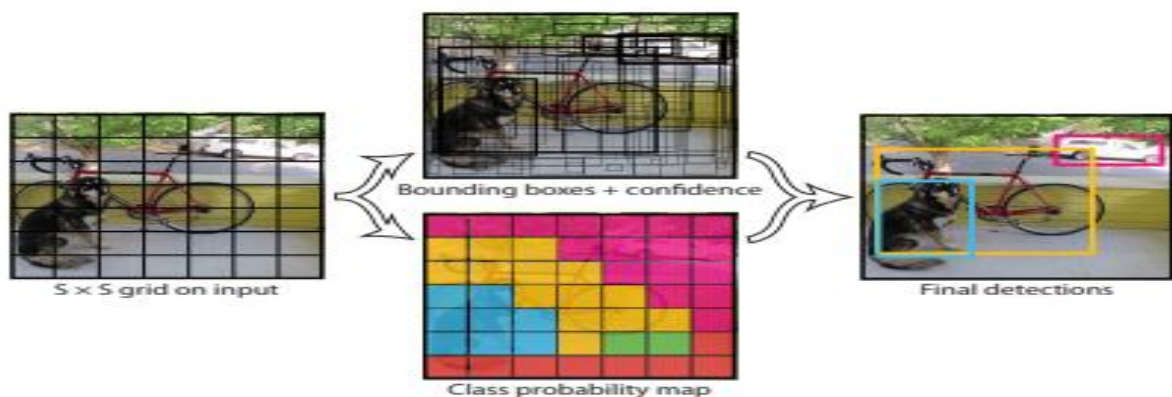


Figure 9 showing how YOLO model work in general [12]

With B number of bounding box predicted per grid, there likely way more bounding boxes than actual object thus NMS is used. NMS is a post-processing step used to remove redundant or incorrect bounding boxes and to output a single bounding box for each object in the image [13]. NMS uses IoU between predicted bounding boxes to remove bounding boxes. IoU is an evaluation metric used in object detection to evaluate performance of object detection by comparing ground truth bounding box to predicted bounding box [14]. If the IoU between 2 predicted bounding boxes are very high, then it means that its likely they are the same object and the one with large IoU is suppressed and discard.

For deep-learning algorithms, we want to be able to extract meaningful representations from the data that best represent the underlying distribution by optimizing the weights and bias of the model. This is the idea of data learning. We do optimization by using backpropagation technique which utilises the gradient descent algorithm. To find the weights/bias that gives the best weight and bias that is representative of the underlying data distribution, we need to minimise the multipart loss function using gradient descent. Multipart loss function indicates how much error the model makes for every prediction, thus by minimising the error between prediction and actual, you essentially get the weight/bias that represent the underlying data distribution and thus you can do MOT with high accuracy.

$$\begin{aligned}
& \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad (3)
\end{aligned}$$

Figure 10: Example of multipart loss function [16]

Multipart loss function (used in YOLOv7 but the equation changes) consists of:

- **Localization or Bounding Box Loss (The first 2 equation):** Penalizes the deviation of predicted bounding box coordinates (x, y, w, h) from the ground truth. Uses sum-squared error for coordinate differences.
- **Confidence Loss (The second 2 equation):** Handles whether the predicted box contains an object (objectness score).
- **Classification Loss (last equation):** Uses sum-squared error for class probability predictions.

The problem in the YOLO is that for each grid it can only output 1 class per grid thus it imposes strong spatial constraints on bounding box predictions and struggles with small objects that appear in groups, such as flocks of birds.

To combat this YOLOv2 introduced key additional components such as anchor boxes, batch normalization, multi-scale training and uses a different backbone called Darknet-19, a variant of VGG-16 [16]. These features are all important to YOLOv7. It is shown that in YOLOv2 that making an architectural change in the backbone can greatly increase accuracy and inference. Anchor boxes are templates used to match objects within a specific feature map. In YOLOv2, these anchor boxes are predefined templates with fixed **sizes** and **aspect ratios** with the same feature map. With Anchor boxes you are now able to predict 5 anchor boxes per grid meaning instead of just 1 class you can essentially predict 5 objects (can be different or similar classes) per grid. This allows for more than 1000 bounding boxes to be predicted compared to the 98 in YOLO [16]. YOLOv2 is now able to make multiple predictions of different classes in that grid.

Batch Normalization is just another layer in training added to the network architecture to stabilise and accelerate training, the intermediate inputs between the layers are normalized [17]. Multi-scale training is implemented by changing the network every few

iterations. Every 10 batches the YOLOv2 network randomly chooses a new image dimension size to train on. This makes YOLOv2 able to detect objects well regardless of input dimension just like YOLOv7.

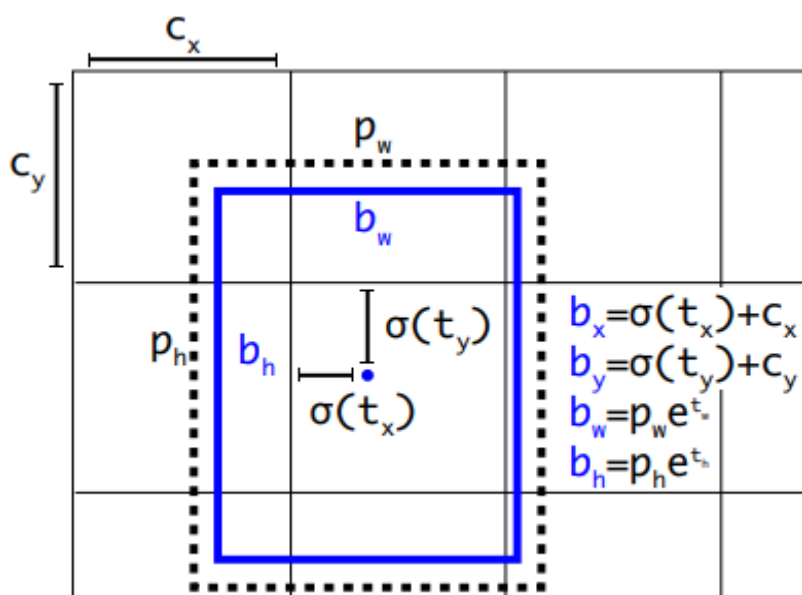


Figure 11: Example of bounding box in YOLOv2 and how its coordinates are calculated [16]

In YOLOv2, anchor boxes were all fixed size and aspect ratio, which limited the ability of the algorithm to detect objects of varied sizes and shapes. It struggles with detecting small objects in high-resolution images or large objects because the feature map resolution limits the granularity of detection. Multi-scale Prediction feature in YOLOv3 allows the algorithm to predict boxes at different scales and aspect ratios. It does this by using something like “Feature Pyramid Networks” whereby the algorithm uses unique features map from different layers. Large objects: Predictions on a low-resolution feature map (52 by 52). Medium objects: Predictions on a medium-resolution feature map (22 x 26). Small objects: Predictions on a high-resolution feature map (13 x 13). We can see that predefined anchor boxes are **assigned to specific feature maps** based on their size. Thus, making it possible to predict the bounding boxes for objects of distinct size to be detected.

In YOLOv2 the confidence (objectness) and classification loss function are no longer the mean squared error which was used YOLOv1. Instead the confidence loss function changed to Binary Cross-Entropy loss, and the classification loss function changed to cross-entropy loss. Both of which are also being used the same as YOLOv7. The Bounding Box Loss in YOLOv2 is completely different from the one use in YOLOv7.

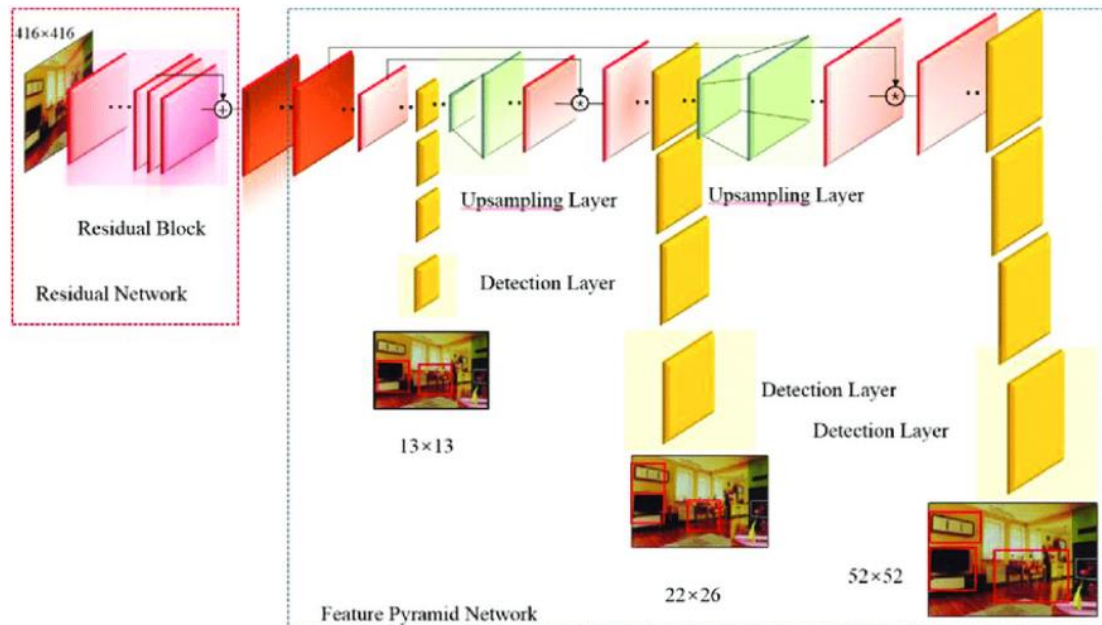


Figure 12: Image displaying how YOLOv3 feature pyramid network using different features map [18]

In YOLOv4, the important features which was used in YOLOv7 is the bag of freebies (BoF) framework which is used for training and the formal introduction of the neck component of the model architecture.

BoF is a set of techniques that improves the model's performance without adding extra computational costs during inference. BoF was used to improve the accuracy of the model's backbone. Data augmentation as well as changing the objective function of the bounding box prediction from the mean squared error to complete intersection over union (CIoU) loss were 2 key method using in BoF[19].

Although CIoU with auxiliary loss head is used in YOLOv7 and not CIoU, the key takeaway is that by changing the loss function, accuracy can improve massively, and inference cost does not increase.

The purpose of data augmentation in BoF is to increase the variability of the input images, so that the designed object detection model has higher robustness to the images obtained from different environments. Real world data often have class imbalance, object diversity issues and limited diversity and thus data augmentation can be used to solve this problem of semantic data distribution.

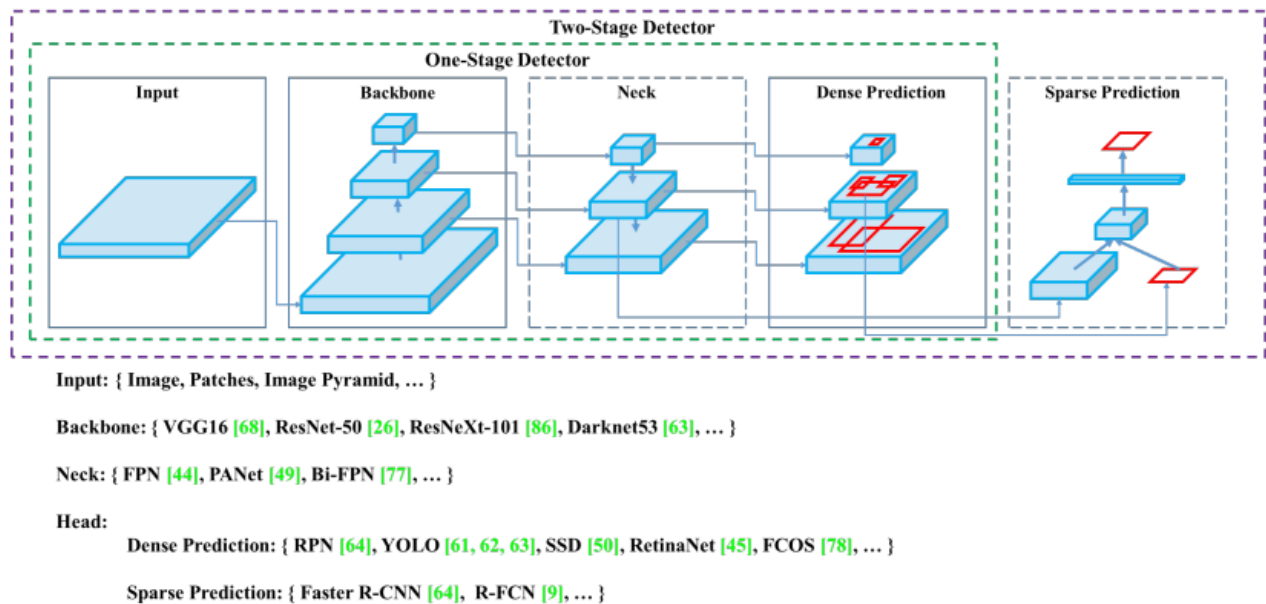


Figure 13: Image showing the neck architecture in YOLOv4 for feature map scaling [19].

The neck component of the model architecture are places where there some layers between backbone and head, and these layers are usually used to collect feature maps from different stages. Just like in the ‘Feature pyramid neck “in YOLOv3, these features map are used with the anchor box to better detect object different sizes as these features map aligns the spatial and semantic information from different scales of an image.

YOLOv5 and v6 were developed by different researchers and companies thus we will not delve into it.

Now we will be delving into YOLOv7. For YOLOv7, it is focusing on real-time object detection. The model architecture they proposed can effectively reduce about 40% parameters and 50% computation of state-of-the-art real-time object detector and has faster inference speed and higher detection accuracy as show in Figure 14 below.

Model	#Param.	FLOPs	Size	AP^{val}	AP_{50}^{val}	AP_{75}^{val}	AP_S^{val}	AP_M^{val}	AP_L^{val}
YOLOv4 [3]	64.4M	142.8G	640	49.7%	68.2%	54.3%	32.9%	54.8%	63.7%
YOLOR-u5 (r6.1) [81]	46.5M	109.1G	640	50.2%	68.7%	54.6%	33.2%	55.5%	63.7%
YOLOv4-CSP [79]	52.9M	120.4G	640	50.3%	68.6%	54.9%	34.2%	55.6%	65.1%
YOLOR-CSP [81]	52.9M	120.4G	640	50.8%	69.5%	55.3%	33.7%	56.0%	65.4%
YOLOv7	36.9M	104.7G	640	51.2%	69.7%	55.5%	35.2%	56.0%	66.7%
improvement	-43%	-15%	-	+0.4	+0.2	+0.2	+1.5	=	+1.3

Figure 14: Comparison the baseline object detectors [19].

FLOPS represents the total number of floating-point arithmetic operations (additions, multiplications, etc.) required to process an input, typically measured per second. In the context of object detection models like YOLO, it refers to the computational cost needed

to process a single image through the network. Thus, it is less computationally expensive to run the detector causing the real-time object detection speed to increase significantly.

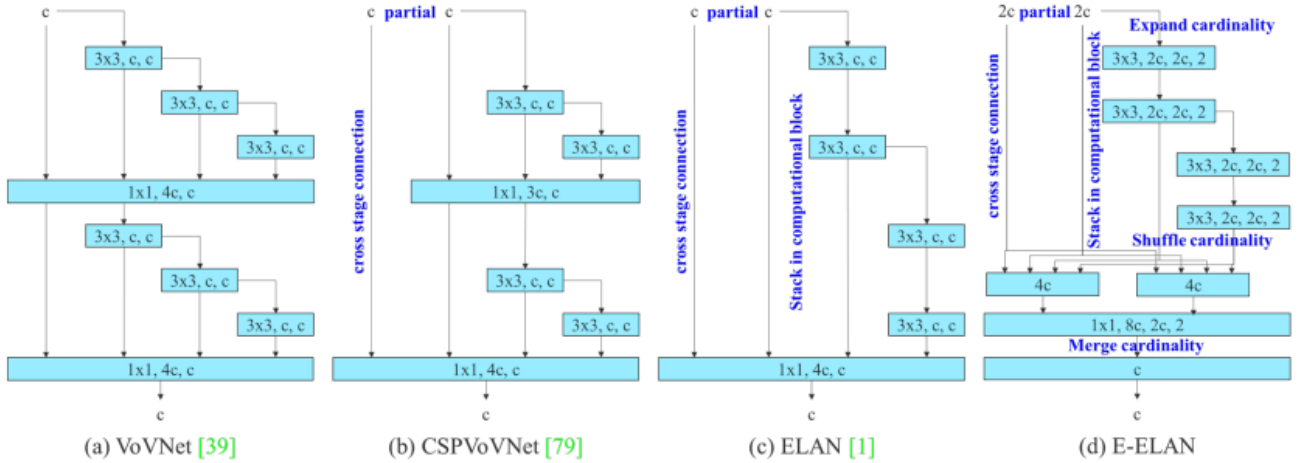


Figure 15: Extended-ELAN architecture used in YOLOv4 [19].

Just like the upgrade from YOLOv1 to YOLOv2; by changing the backbone network architecture, we can greatly improve the detection accuracies. Thus, in YOLOv7, by upgrading the model architecture to the Extended efficient layer aggregation network (E-ELAN) architecture for the backbone network, it allows for better detection accuracies.

The E-ELAN architecture has uninterrupted gradient flow (easier to update weight/bias of model) through the network, allowing deeper networks to train efficiently. The architecture unifies feature extraction, aggregation, and prediction into an end-to-end flow, which is optimized for real-time object detection.

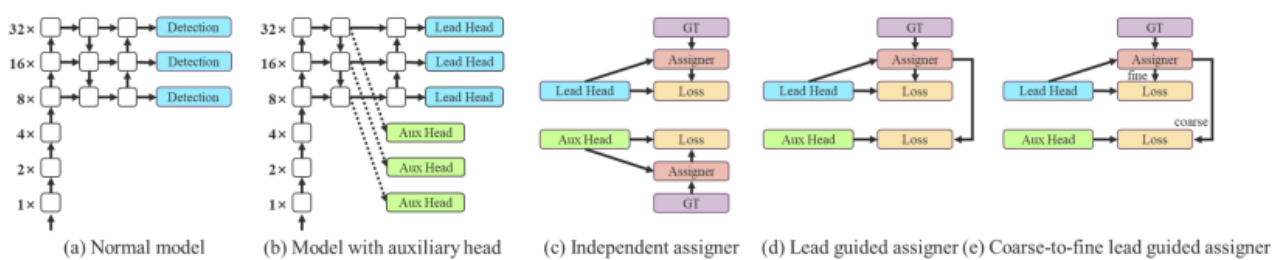


Figure 16: CloU loss function with auxiliary supervision [19]

Deep supervision is a training technique used for training deep networks [19]. In YOLOv7, the CloU loss function is accompanied by the auxiliary supervision. It works by adding extra auxiliary head in the middle layers of the network, and the shallow network weights with CloU loss on the bounding box as the guide. This ensures that intermediate layers are explicitly trained to align bounding boxes effectively, even before the final output layer refines them.

In addition to all the key components from previous model architectures as well as newer component we would get YOLOv7. This deep supervision technique part of the trainable bag of freebies that is used in YOLOv7. Other trainable BoF methods that were used is model re-parameterization (at model/module level) as well as model scaling. Model scaling is the idea of being able to essentially convert the YOLOv7 model into different size to fit different computing platforms, one example is YOLOv7-tiny being meant for computing platforms with very little computation. Model re-parameterization is the idea of training individual models and averaging their weights to get the best model result. These are all possible due to YOLOv7 using E-ELAN as its backbone.

3.2.2 ByteTrack Algorithm (Object Tracking)

After we've covered object detection, we now move on to the next important step: object tracking, specifically using the ByteTrack algorithm. Object tracking is a crucial part of the process because once objects are initially detected, they often move around, and multiple objects can sometimes overlap.

Understanding track, tracklets, Re-identification (Re-ID) and Re-ID features for Object tracking

- Track refers to the entire trajectory of a unique object's path.
- Tracklet is short track of a few frames generally.
- Re-ID is the process of recognizing and associating the same object across different frames or scenes, even if it disappears temporarily or moves into a different area of the camera's field of vision.
- Re-ID features are representations extracted from the objects identified in the current frame to capture its appearance and identity. They are represented by high-dimensional vectors in both FairMOT and ByteTrack

ByteTrack, a novel approach to multi-object tracking (MOT) is especially helpful in complicated, real-world situations. In crowded scenes, objects can overlap, be partially detected, or even disappear briefly. Traditional tracking methods struggle in these cases and may lose track of objects when they become occluded or hard to see.

Traditional tracking methods like SORT, DEEPSORT and more, struggle in complex environment because they only keep the high score detection boxes. However, Byte Track, keeps almost every detection box and separate them into high score ones and low score ones. Low Detection Scores are very common in real-life scenarios as people will tend to overlap in crowded scenes, motion blur is very common in picture, the lighting might not be right. Thus, when tracking in these complex environments which are common, it will predict a lot of the low detection score bounding boxes, this will cause most of these data to be removed and rendered useless if their score is under a certain threshold in traditional tracking methods.

How Bytetrack works in general [8]:

1. **High-Confidence Association:** ByteTrack starts by focusing on high-confidence detection boxes. These boxes have a high probability of containing an object. In this first step, ByteTrack links these confident detections to set up a strong baseline track for each object. By starting with high-confidence boxes, ByteTrack ensures that the initial tracks are reliable and clear. Some tracklets get unmatched because they do not match to an appropriate high score detection box, which usually happens when occlusion, motion blur or size changing occurs.
2. **Low-Confidence Association:** Once the high-confidence detections are linked, ByteTrack moves on to the low-confidence detections. Low-confidence detections may happen when objects are partially visible, blurred, or briefly out of view. ByteTrack checks these detections and associates them with existing tracks only if they fit reasonably well. This way, even less confident detections are used, so no object is left behind. The low score detection boxes and these unmatched tracklets is able recover the objects in low score detection boxes and filter out background, simultaneously.

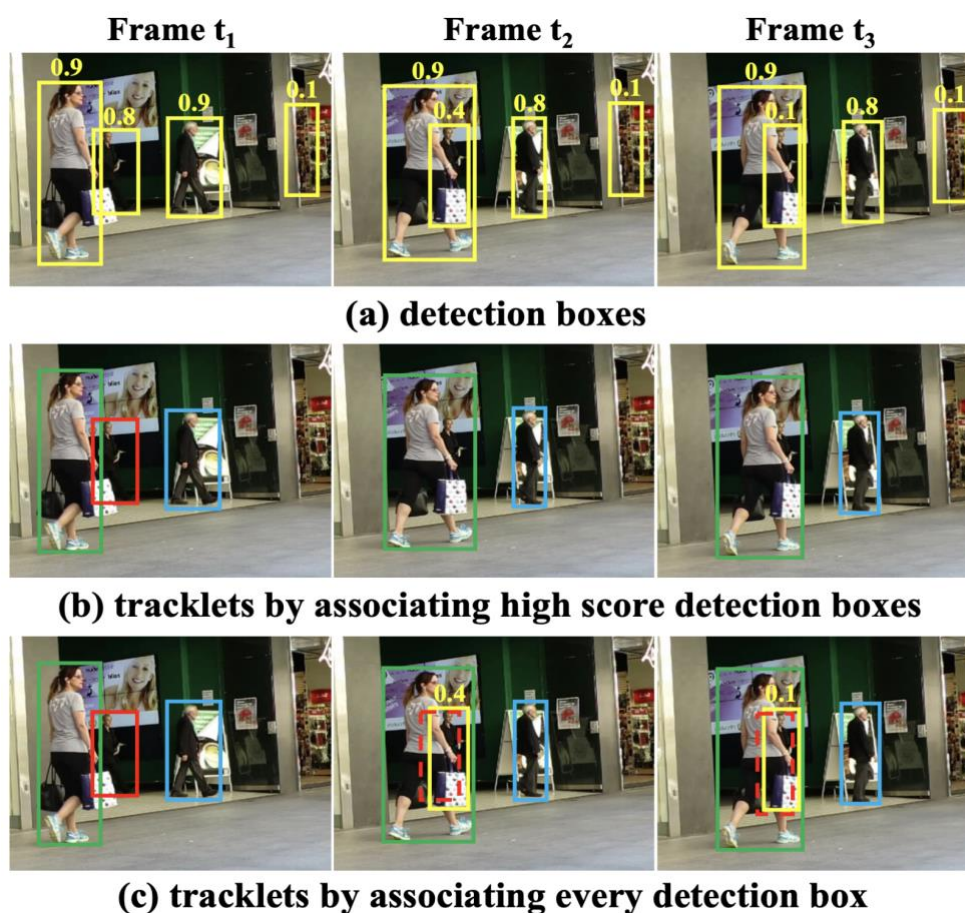


Figure 17: How ByteTrack keeps low confidence boxes during association [8]

We can see from Figure 17 above in (c) that even if there is very high overlap and the detection score is very low, the bounding box is still able to track the person with the ByteTrack algorithm whereas in (b) where only high score detection boxes are kept, it is

unable to track the person properly, demonstrating the relevance of the ByteTrack algorithm in real-life scenarios.

3.2.2a Core of ByteTrack (BYTE data association method)

For MOT, data association is a core component. It works by computing the similarity between tracklets and detection boxes then leveraging different strategies to match them according to their similarity.

The BYTE algorithm (You can refer to the image below) [23]:

1. Input: Video sequence, Object detector (YOLOv7), Detection Score Threshold
2. Detection boxes and boxes are computed by the object detector. The detection score threshold segregates the detection boxes into high-confidence and low-confidence detection boxes using their detection score. Kalman filter (motion model) is used for predicting the next position of each tracklets based on previous state tracks. In BYTE, it is used for associating tracks with detections.
3. Two-step Association Process:
 - a. **High-Confidence Association:** High-confidence association is done between all the track and high confidence detection box using the similarity value. The similarity value is computed via IoU or Re-ID features distance of all predicted box of tracks and the high confidence boxes. The Hungarian Algorithm does this matching/association. The unmatched tracks and high-confidence detection boxes are still kept
 - b. **Low-Confidence Association:** The unmatched confidence detection boxes is combined with the low-confidence detection boxes. They are then used to do a second association with the unmatched tracks. The remaining unmatched low-confidence detection boxes are deleted
4. All unmatched tracks after both association steps are now deleted.
5. We will now do this BYTE algorithm recursively until the video ends or once we stop streaming in video sequences on the robot.

The BYTE algorithm can work effectively in crowded and complex environments as it allows the system to continue tracking objects accurately even when they overlap, become hard to see, or are partially blocked from view. ByteTrack's approach makes it particularly reliable in scenarios where objects are constantly moving, disappearing, and reappearing.

A unique point to note about ByteTrack algorithm is that it can work with different type of detectors (be it 2 stage or 1 stage detectors) and different types of similarity value can

be used for association (Re-ID features, IoU etc) as it is just a tracking algorithm itself. In the paper it uses YOLOv5 as the detector but in our project, we used YOLOv7 as it is faster, more efficient and accurate than YOLOv5 and more stable than the newer version of the YOLO beyond version 7.

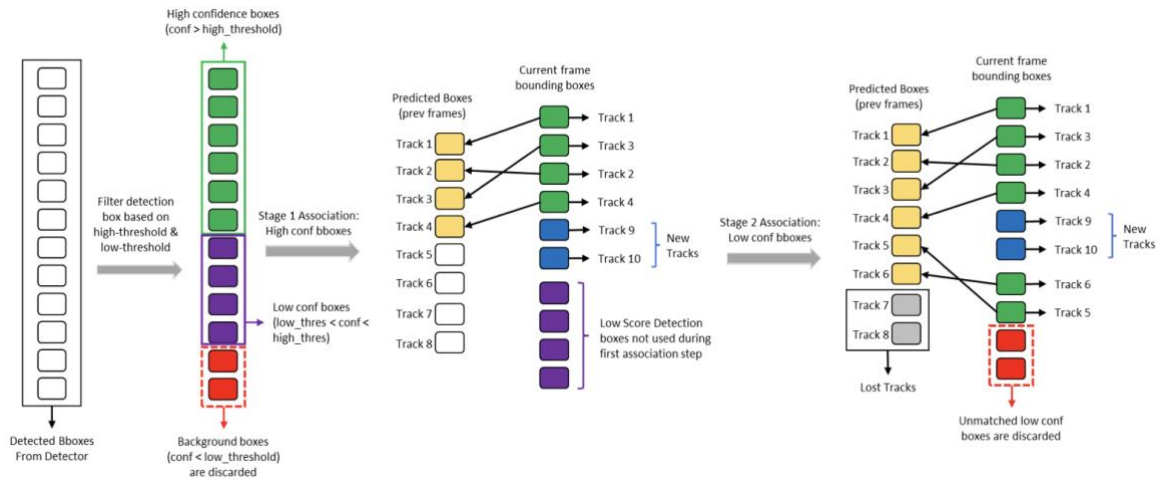


Figure 18: General visualization how Byte Algorithm works [23]

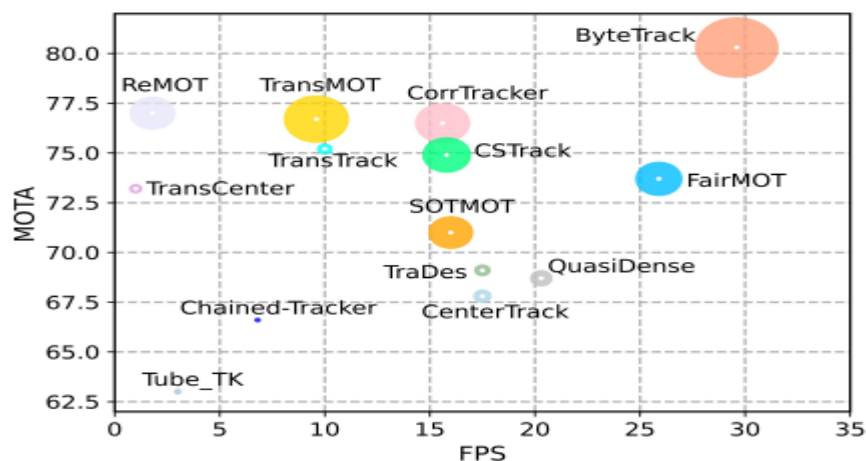


Figure 19: Results of Byte Algorithm [8]

We can see from this graph that for the MOTA dataset, for high frames per second ByteTrack can perform well. Simply put this means that it performs much faster than comparable tracking algorithms.

3.2.3 YOLOv7 + ByteTrack Results

We trained the YOLOv7 + ByteTrack model with our own dataset consisting over well over 33000 images. We ran the model on a Nvidia GeForce RTX 3090. We trained it for 60 epochs (1 epoch means 1 training run of 33000 images) with a batch size of 24 for around 12 hours. While the original YOLOv7 was trained on the COCO dataset with more than 330000 images, we still managed to achieve state-of-the-art results.

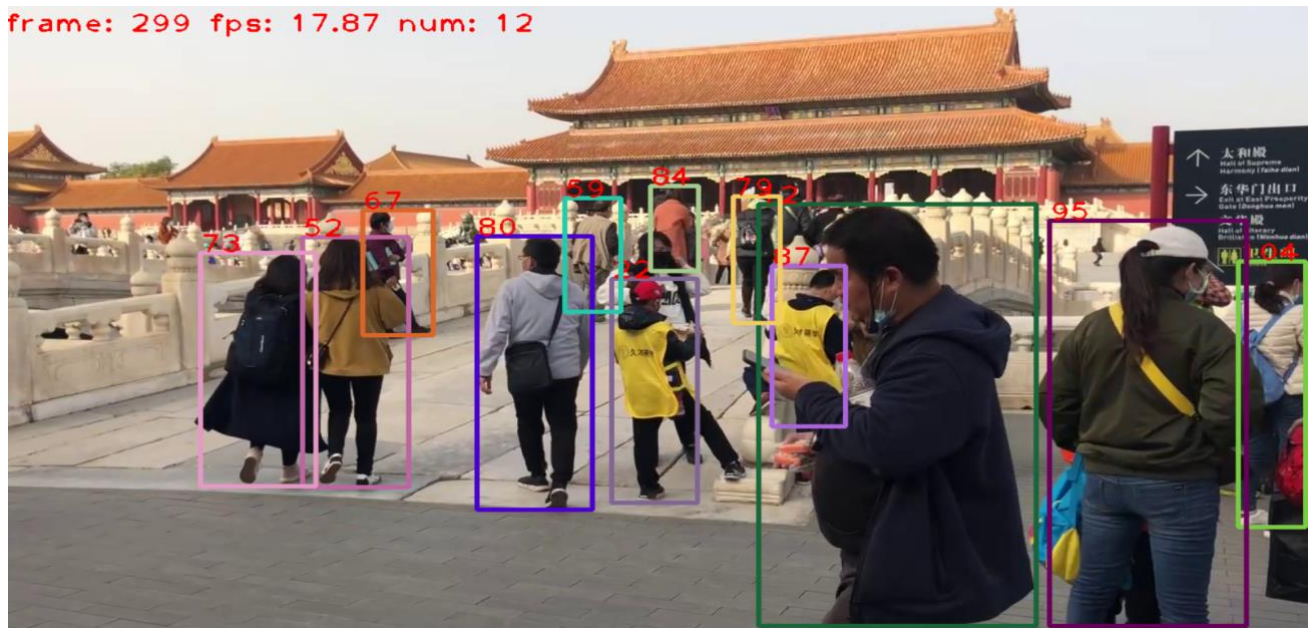


Figure 20: YOLOv7 on the demo video.

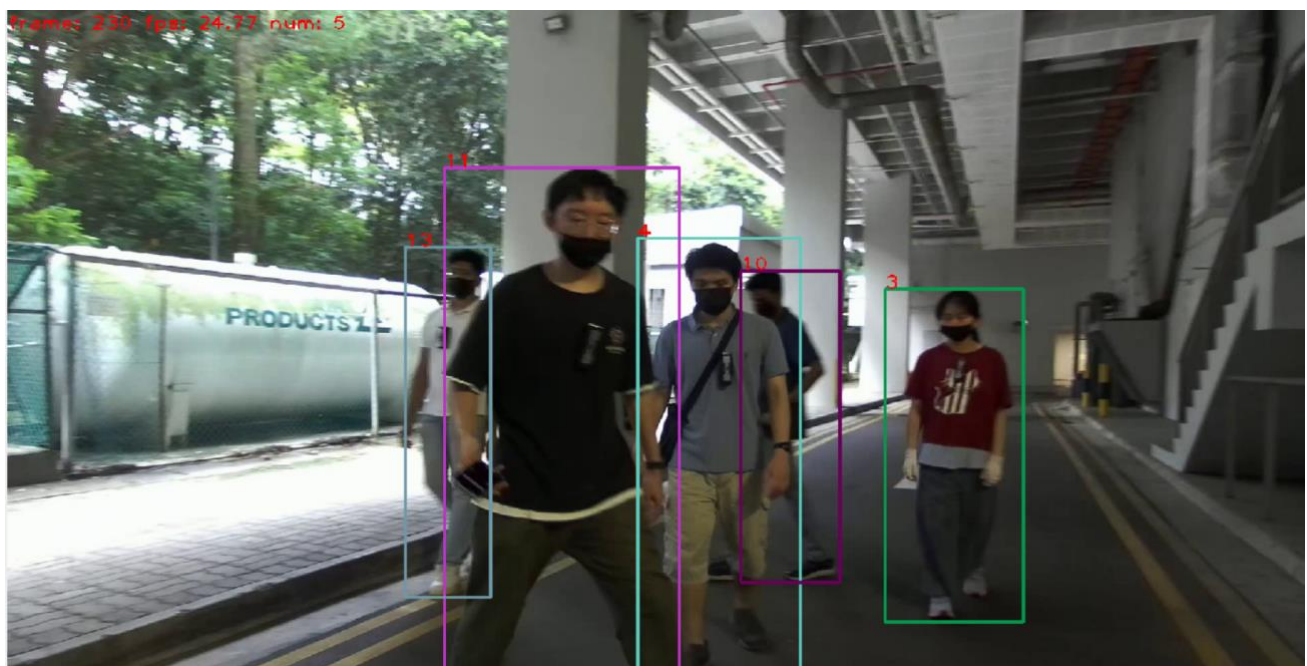


Figure 21: YOLOv7+Bytetrack on our test video which was recorded on the moving robot.

We can see from both videos quite clearly that our trained model is able to working accurately on the demo video and on our own test set.

The 2 metrics we are focusing on is Multiple Object Tracking Accuracy (MOTA) and IDF1.

MOTA (as shown below) is a metric which measures the overall accuracy of a multi-object tracking system. It gives a single score to reflect the overall tracking performance

by evaluating how well the tracker performs when we combine three major error types: **false positives (FP)**, **false negatives (FN)**, and **identity switches (IDs)**.

The MOTA metric is computed across all frames as:

$$\text{MOTA} = 1 - \frac{\sum_t (FN_t + FP_t + IDSW_t)}{\sum_t GT_t}$$

Where t is the frame index, FN are False Negatives, FP are False Positives, IDSW are Identity Switches, and GT is the number of ground truth objects.

Figure 22: MOTA Metric Computation [20]

IDF1 is based on Identification Recall and Identification Precision. It computes the assignment between prediction and ground truth objects across the entire video, rather than frame by frame.

$$\text{IDF1} = \frac{2 * \text{IDTP}}{2 * \text{IDTP} + \text{IDFP} + \text{IDFN}}$$

Where:

- **IDTP** (ID True Positive): The number of correctly identified detections
- **IDFP** (ID False Positive): Tracker predictions that don't match any ground truth
- **IDFN** (ID False Negative): Ground truth trajectories that aren't tracked

Figure 23: IDF Computation [20]

DIP yolov7+bytetrack_final

	IDF1	IDP	IDR	<u>Rc11</u>	<u>Pr11</u>	GT	MT	PT	ML	FP	FN	IDs	FM	MOTA	MOTP	<u>IDt</u>	<u>IDa</u>	<u>IDm</u>
test2	65.3%	66.0%	64.6%	98.8%	99.9%	4	4	0	0	10	80	5	15	98.7%	0.065	1	6	0
test5	56.2%	57.0%	55.4%	97.8%	99.5%	6	6	0	0	20	200	20	60	97.1%	0.070	5	15	0
test8	60.0%	60.7%	59.3%	96.5%	98.5%	8	8	0	0	100	300	40	80	96.0%	0.075	6	25	0
test10	55.5%	55.7%	55.3%	98.0%	98.2%	4	4	0	0	130	150	15	40	96.2%	0.067	2	10	0
test13	54.9%	55.0%	54.7%	97.2%	97.0%	6	6	0	0	200	250	30	65	94.8%	0.075	4	20	0
test16	50.0%	50.5%	49.6%	94.7%	96.8%	8	8	0	0	220	564	60	100	92.8%	0.080	11	40	0
OVERALL	57.7%	58.2%	57.2%	97.5%	98.7%	36	36	0	0	680	1644	170	360	96.3%	0.072	29	116	0

Figure 24: YOLOv7 ByteTrack Results

We can see that our MOTA score is around 96.3% and IDF1 score is around 57.7%. While our MOTA score manages to outperform the original byte track algorithm of 80% around IDF1 results performs around the same relative to state-of-the-art results [20].

3.2.3.1 Introducing the 1-stage/1-shot Approach Fair-MOT

Following Our Exploration of the 1 stage YOLOv7 + ByteTrack algorithm, we will now look at the FairMOT algorithm for Multi-Object Tracking. FairMOT is unique because it combines object detection and tracking together in a single, unified framework, which helps in balancing tracking accuracy across all detected objects without introducing any bias. This approach is designed to ensure that both identity preservation and spatial tracking are handled effectively, even when we're dealing with objects that move at different speeds or are packed closely together in dense environments. FairMOT is especially useful for maintaining consistent identity tracking, helping to minimize the risk of identity switches that can occur when objects overlap or are positioned very close to each other.

3.3.3.2 What might cause FairMOT to fail (Joint Object Detection + Object Tracking)

Before we delve into the architecture of FairMOT, we will first introduce why one-shot trackers might not work. For one-shot trackers, there are 3 main problems which FairMOT resolves:

1. Unfairness caused by anchor boxes: When using an anchor-based design, anchors does allow for very accurate detection however they also result in large amounts of ID switching. This means that the ID of the tracked object does not stay the same
2. Unfairness caused by sharing features between object detection and tracking. For most one-shot trackers Object detection and object tracking share the same features, however this is no desirable as they are different task
3. Unfairness cause by feature dimensions. While high feature dimensions help with object detection, low feature dimensions allow for faster inference and better results for object tracking

In the next section, we will explain how these unfairnesses would be resolved

3.2.3.3 Innerworkings of Fair-MOT (Joint Object Detection + Object Tracking)

FairMOT Backbone Encoder-Decoder Neural Network

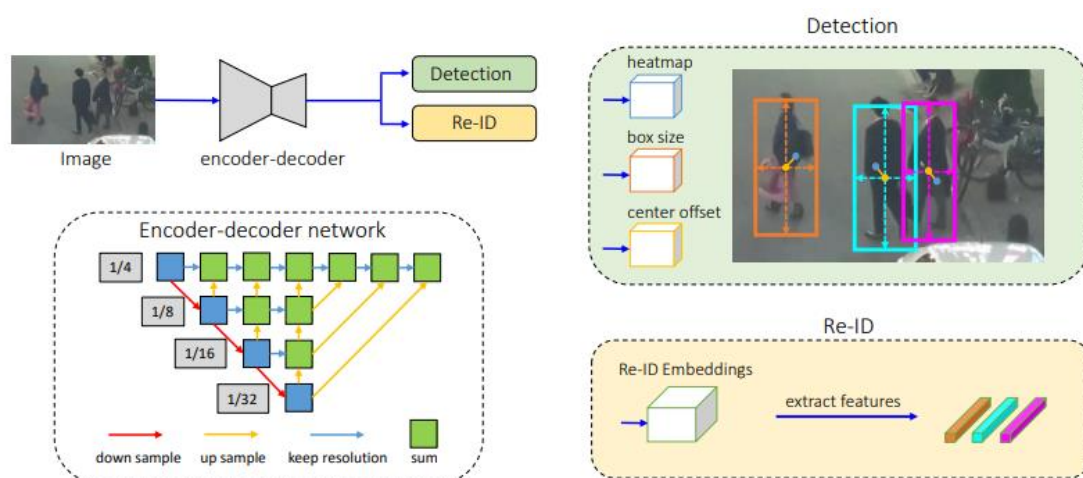


Figure 25: General Architecture of the FairMOT model [23]

The Backbone used is DLA-34 which acts feature extractor which provides fair high resolution features maps for both the Detection and Re-ID branches [23]. High resolution means these feature maps keeps a very high amount of spatial information about the input images. These feature maps are considered “fair” as they are multi-scale high resolution feature maps that are extracted at different scales. This results in feature maps that capture the diverse size and appearance of objects, making them versatile for both detection and Re-ID task.

FairMOT is a one-shot tracker as the detection and Re-ID branches happen simultaneously. Model run the detection branch, which is used for detecting object, and Re-ID branch which is used for extracting high-level features [23].

FairMOT Detection branch

The detection head of FairMOT uses CenterNet [23], an anchor-less detector, which resolves the unfairness cause by anchor-based approaches. It contains the Heatmap head, and the box offset and size head

1. Heatmap head is used to precisely estimate the locations of the object centers
2. Box offset head aims to precisely localize object and size head's aim is to precisely estimate the height and width of the object

Loss function of Heatmap Head

The loss function used by the heatmap head is called the pixel-wise logistic regression focal loss function as shown below

$$L_{\text{heat}} = -\frac{1}{N} \sum_{xy} \begin{cases} (1 - \hat{M}_{xy})^{\alpha} \log(\hat{M}_{xy}), & M_{xy} = 1; \\ (1 - M_{xy})^{\beta} (\hat{M}_{xy})^{\alpha} \log(1 - \hat{M}_{xy}) & \text{otherwise,} \end{cases} \quad (1)$$

Figure 26: Loss function used for heatmap Head [23]

The heatmap loss function treat every pixel as a binary classification problem which ensures that the detector accurately identify centers by reducing the error between the pixels. However, since most pixels are just backgrounds thus it is easy to predict all pixels as background. This issue is resolved by the focal loss aspect of the loss function reduces the imbalance between easy negatives (background pixels) and hard positives (object centers). This would allow the detector to detect even the smallest of objects as the error rate of the center would be higher if it's not detected causing the weight and bias of the model to be adjusted accordingly.

Loss function of Box Offset and Size Head

$$L_{\text{box}} = \sum_{i=1}^N \|\mathbf{o}^i - \hat{\mathbf{o}}^i\|_1 + \lambda_s \|\mathbf{s}^i - \hat{\mathbf{s}}^i\|_1.$$

Figure 27: Loss function used for Box Offset and Size Head [23]

In this case represent the offset value, S represents the size value. $\hat{O}$ is the predicted offset value and $\hat{S}$ represents the predicted size value.

The box offset and size head loss function ensures that the predicted bounding box align very closely with the actual objects in the image by reducing the error between the predicted and actual values and updating the weights to best represent

FairMOT Re-ID branch

The Re-ID branch treats the process of Re-ID as a classification task. The predicted output vector should be closely align the output vector of a certain class. The loss function ensures that the embeddings are class-specific and encourages the network to assign a high probability to the correct identity while suppressing others.

$$L_{\text{identity}} = - \sum_{i=1}^N \sum_{k=1}^K \mathbf{L}^i(k) \log(\mathbf{p}(k)),$$

Figure 28: Loss function used for Re-ID Head [23]

K is the number of all the identities in the training data.

FairMOT final uncertainty Loss Function

$$L_{\text{detection}} = L_{\text{heat}} + L_{\text{box}}, \quad (4)$$

$$L_{\text{total}} = \frac{1}{2} \left(\frac{1}{e^{w_1}} L_{\text{detection}} + \frac{1}{e^{w_2}} L_{\text{identity}} + w_1 + w_2 \right), \quad (5)$$

Figure 29: Loss Function used for uncertainty Loss [23]

The final loss function is called the uncertainty loss function which combine the Re-ID loss and the Detection loss functions. This loss function balances the unfairness caused by the difference in feature dimensions between the Re-ID and detection branch. It would also balance the unfairness caused by sharing the of high-resolution shared fair feature maps in the encoder-decoder backbone. It achieves the automatic balancing feature as w_1 and w_2 are learnable parameters that would adjust the relative contributions of the Re-ID and detection branches to be fair during the training process.

3.2.3.2 Evaluation of FairMOT



Figure 30: Our trained FairMOT working on a demo video

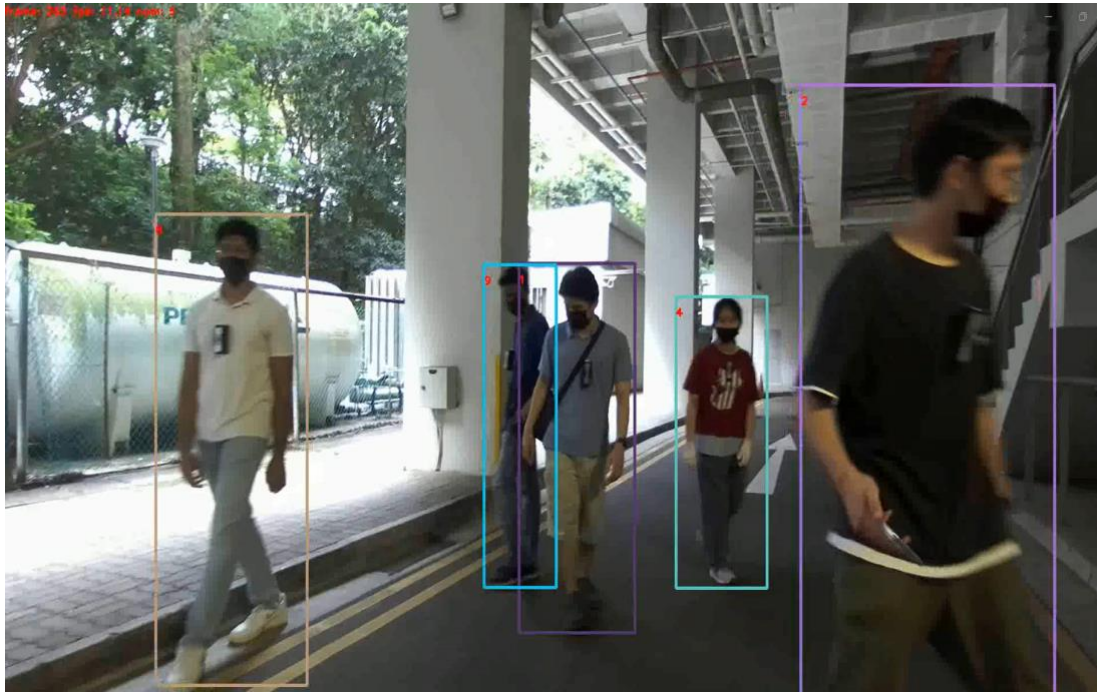


Figure 31: FairMOT working on the test video recorded on moving robot

	IDF1	IDP	IDR	Rc1l	Prcn	GT	MT	PT	ML	FP	FN	IDs	FM	MOTA	MOTP	IDt	IDa	IDm
test2	87.4%	87.3%	87.5%	99.5%	99.2%	4	4	0	0	58	37	8	17	98.6%	0.085	0	3	0
test5	64.8%	64.8%	64.7%	98.9%	99.0%	6	6	0	0	94	103	14	36	97.7%	0.086	2	8	0
test8	63.7%	63.4%	63.9%	98.7%	98.0%	8	8	0	0	220	141	33	59	96.3%	0.095	8	19	0
test11	90.0%	90.0%	89.9%	99.4%	99.5%	4	4	0	0	41	50	4	24	98.8%	0.090	0	3	0
test14	62.9%	63.1%	62.7%	98.7%	99.3%	6	6	0	0	57	112	14	45	97.9%	0.096	5	9	0
test17	51.6%	51.7%	51.5%	98.2%	98.7%	8	8	0	0	149	200	29	62	96.6%	0.104	2	23	0
OVERALL	68.2%	68.2%	68.2%	98.8%	98.9%	36	36	0	0	619	643	102	243	97.5%	0.094	17	65	0

Figure 32: FairMOT’s peformance on our testset

As you can see from above, the IDF1 score of 68.2% and MOTA score of 97.5%. Our Trained FairMOT model were able to outperform our own YOLOv7 + byte track model on our test set. However, we can see that the average FPS is 9-11 FPS on the FairMOT model itself which is terrible compared to YOLOv7 + byte track of 18-25 FPS.

3.3 Implementation of the YOLOv7 + Bytetrack Algorithm on the Robot

How the robot works:

We used the robot as shown in Figure 33 below for multi-object tracking. The robot is equipped with wheels, ZED 2 AI stereo cameras are installed on the top and four antennae installed at the front as well as a portable battery (which is installed above the wheels) for it to move freely. The operating system the robot is running on is Ubuntu 20.04.6. With this robot, it allows us to test the YOLOv7 + ByteTrack model for object tracking and localization in challenging and complex environments.

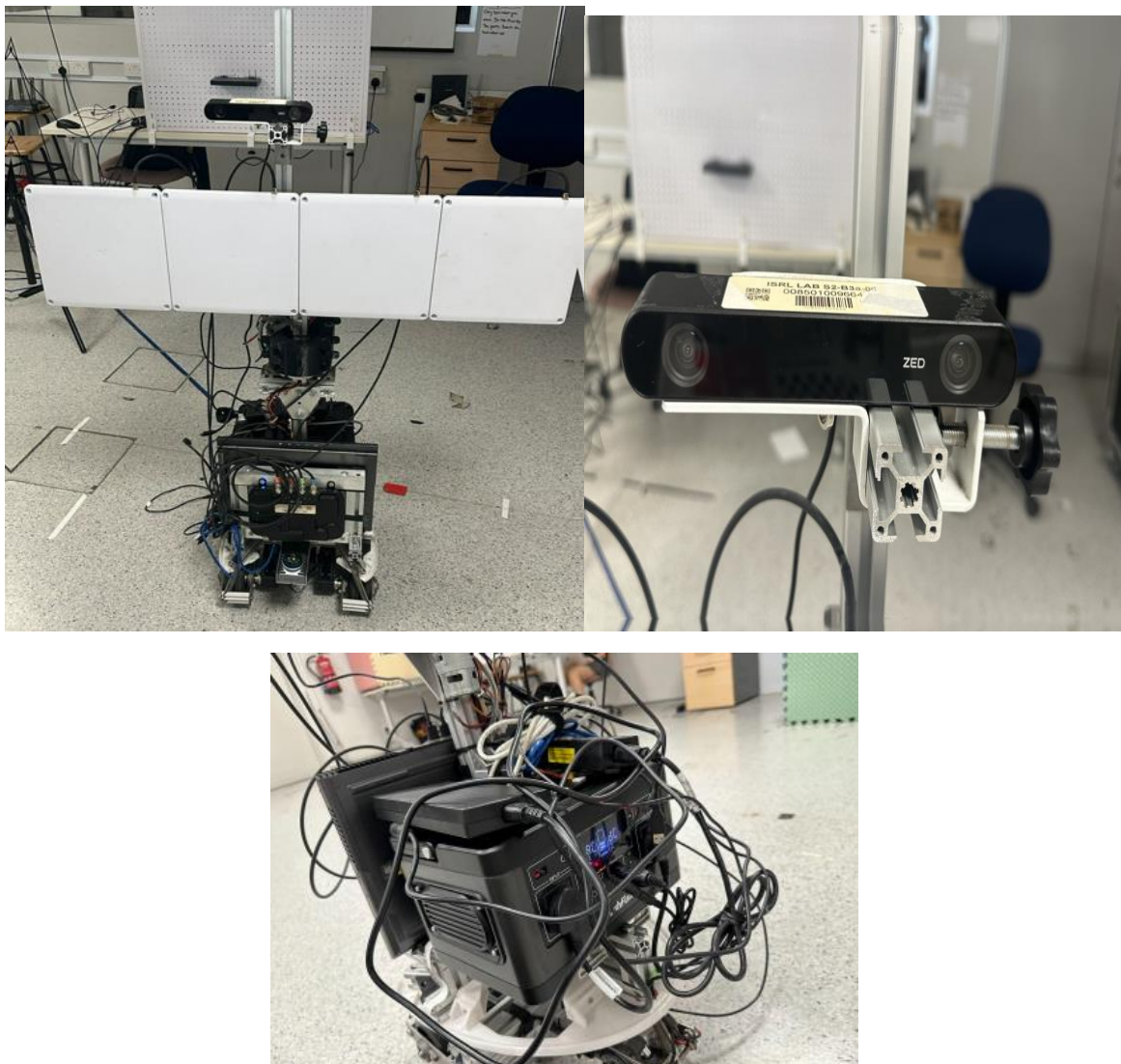


Figure 33: Images of the experiment robot, with close-up shots of ZED and RealSense depth cameras and the portable battery.

Robot Operating System (ROS)

To control the robot and its movements around the complex environment it is situated in, we used ROS. ROS is an opensource middleware suite built for robotic application. It is a set of software libraries and tools that allows us to build robot applications. Most of the robots operating today use ROS and it is an industry standard. ROS is supported by and built on top of Ubuntu Linux which our robot is running. The ROS distribution that we are using is ROS 1 Noetic. With ROS there is not a need for us to reprogram the way the low-level functionalities of the Robot which includes actuators, motor controllers, or sensor drivers for every project. We can just focus on high level task like AI-integration, object detection and many more. This is critical for us due to the time-constraints of this DIP project and the focus is to integrate AI models not to build a robot from scratch.

Understanding ROS

- **ROS 1 Noetic** utilises a **master-slave architecture**.
- **ROS master** manages the communication between nodes.
- **ROS nodes** are a single-purpose executable program that does a certain task and outputs a certain stream of message
- **ROS topics** are how nodes communicate with each other, a topic is just a stream of messages (which can be any form of data) transferred between nodes. ROS nodes can publish/subscribe to these topics. When you publish the messages to the topic, any ROS node that subscribed to that topic is able to receive the message
- **ROS Messages** are a data structure representing that define the topic

ROS nodes can pass messages to one another through topics, make service calls to other nodes, provide a service for other nodes, or set or retrieve shared data from a communal database called the parameter server.

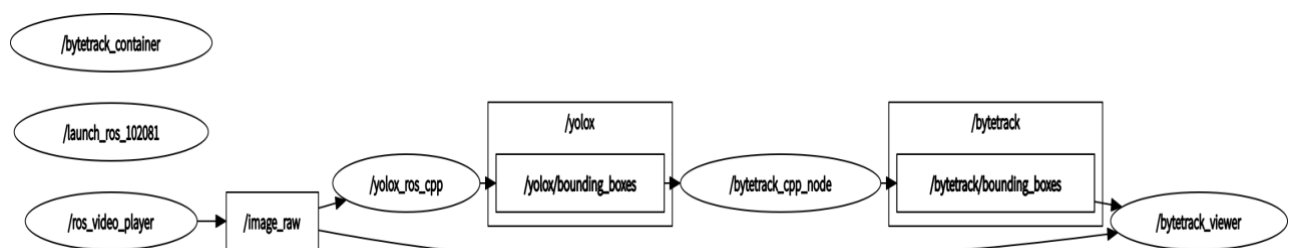


Figure 34: Example ROS graph to implement [21]

The above example (Figure 34) ROS graph is how we are implementing YOLOv7+ ByteTrack, the only that is difference would be that the detector we are using is YOLOv7 instead of YOLOx.

What we used to program the robot

Rospy, a Python interface for ROS, enabling seamless communication and data exchange between the ROS environment and our Python-based algorithms. With rospy, it enabled us to program a ROS node to control the robot while integrating the YOLOv7 + ByteTrack model for tracking.

How the robot has been programmed to work

The robot will essentially be tracking the first person that moves into the frames. We then have multiple people come into the frame to distract the robot and track another person. However, the robot should not deviate from that person and will continue following that person. The reason why we have multiple people walk into the frame is that we want to simulate a challenging situation of a crowded place. This demonstrates the effectiveness of the YOLOv7 + Byte track model's MOT abilities as even in the midst of a complex situations the model can track the person.

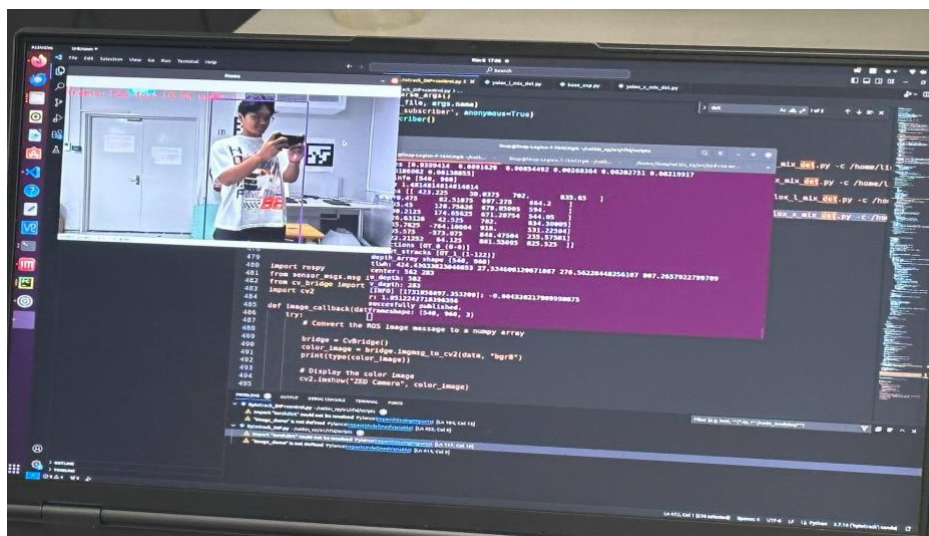


Figure 35: Showing the robot working on a static target

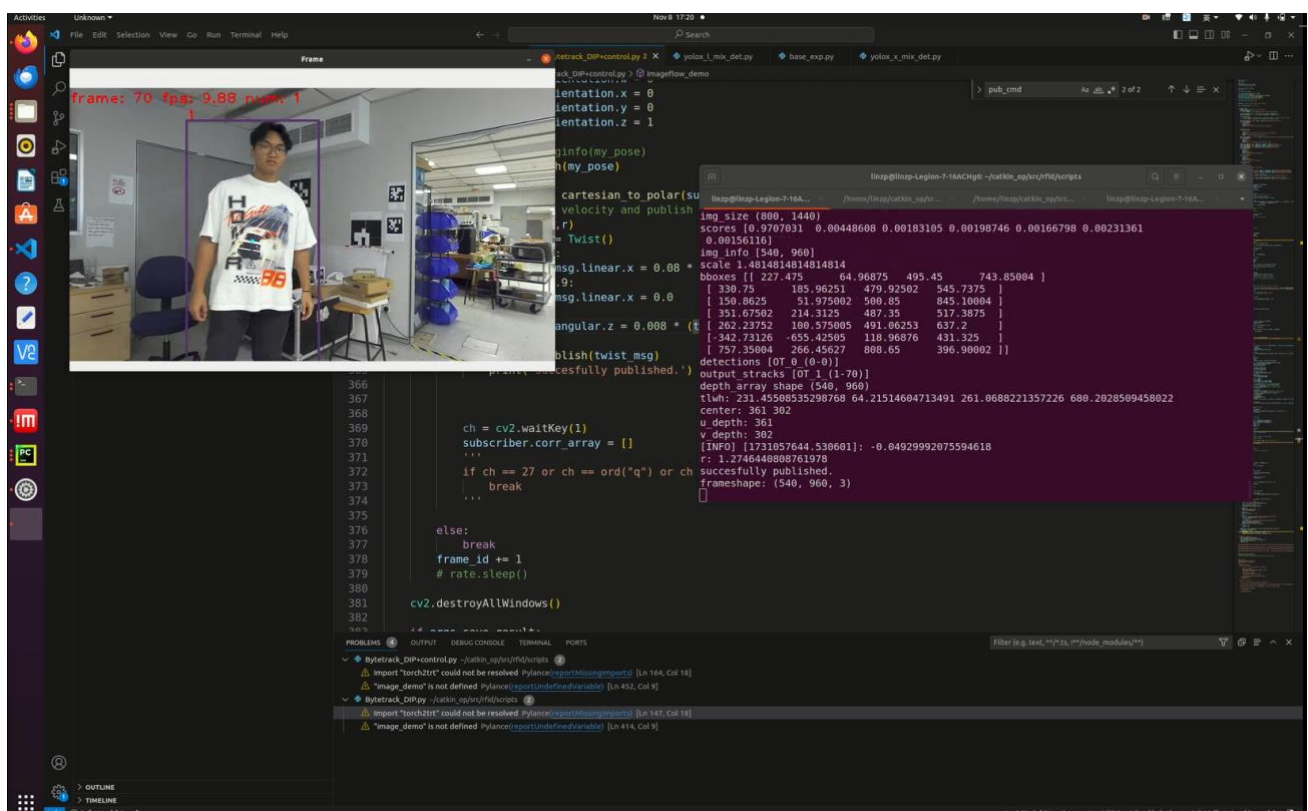


Figure 36: The Image should depict the robot tracking a moving person from the CV team

Entire Process of How the Robot Works using our ROS node program

Step 1: Input Data acquisition (input video frame and depth data)

There is a ROS ZED node whereby the ZED2 camera publishes streams of camera images and depth data to 2 ROS topics in real time.

```

class ImageSubscriber:
    def __init__(self):
        self.cv_image = None
        self.depth_image = None
        self.corr_array = [] # save the 3D coordinates of pixels on the image
        self.bridge = CvBridge()
        self.image_sub = rospy.Subscriber('/zed2i/zed_node/rgb/image_rect_color', Image, self.image_callback)
        self.depth_sub = rospy.Subscriber('/zed2i/zed_node/depth/depth_registered', Image, self.depth_data_callback)

    def image_callback(self, data):
        try:
            self.cv_image = self.bridge.imgmsg_to_cv2(data, "bgr8")
        except Exception as e:
            print(e)

    def depth_data_callback(self, data):
        try:
            self.depth_image = self.bridge.imgmsg_to_cv2(data, "32FC1")
        except Exception as e:
            print(e)

```

Figure 37: Depiction of Data Acquisition

The ImageSubscriber class is used to subscribe to the 2 ROS topics to get the camera input as well as the depth data. The image data would be used by the CV model to track the target. The depth data, which provides the distance of each pixel in the frame from the camera lens, is of particular importance as we want the robot to be able to track and follow the target.

Step 2: Do the detection and tracking of target using YOLOv7 + ByteTrack

```

while not rospy.is_shutdown():
    if frame_id % 20 == 0:
        logger.info('Processing frame {} ({:.2f} fps)'.format(frame_id, 1. / max(1e-5, timer.average_time)))
    if subscriber.cv_image is not None:
        frame = subscriber.cv_image
        print('frameshape:', frame.shape)
        outputs, img_info = predictor.inference(frame, timer)
        # print("outputs:", outputs)
        #function
        if outputs[0] is not None:
            online_targets = tracker.update(outputs[0], [img_info['height'], img_info['width']], exp.test_size)
            online_tlwhs = []
            online_ids = []
            online_scores = []
            for t in online_targets:
                tlwh = t.tlwh
                tid = t.track_id
                vertical = tlwh[2] / tlwh[3] > args.aspect_ratio_thresh

                depth_array = np.array(subscriber.depth_image, dtype=np.float32)
                print('depth_array shape:', depth_array.shape)
                if int(tlwh[0] + tlwh[2]) > width:
                    u = int((tlwh[0] + width) / 2)
                else:
                    u = int((tlwh[0] + tlwh[2]) / 2)
                if int(tlwh[1] + tlwh[3]) > height:
                    v = int((tlwh[1] + height) / 2)
                else:
                    v = int((tlwh[1] + tlwh[3]) / 2)
                print('tlwh:', tlwh[0], tlwh[1], tlwh[2], tlwh[3])
                print('center:', u, v)
                centers.append([u, v])
                u_depth = int(u * depth_width / width)
                v_depth = int(v * depth_height / height)

                print('u_depth:', u_depth)
                print('v_depth:', v_depth)
                if u_depth < depth_width and v < depth_height:
                    depth_value = depth_array[v_depth, u_depth]

```

Figure 38: Representation of detection and tracking of target using YOLOv7 + ByteTrack

The “predictor” represents the YOLOv7 model to detect objects provides the bounding boxes to the “tracker” which represents the ByteTrack Algorithm to associate detections across frames and track the target to maintains object identities and updates bounding boxes. The bounding boxes contains the x and y co-ordinates of the center of the tracked object. This way the robot can follow and move according to the tracked object’s movements.

Step 3: Position Calculation of the tracked target

We need to first utilise the 2D image + depth data to calculate the 3D world coordinates for tracked target. By converting the center of object in the 2D image space with depth data to its corresponding 3D World Coordinates.

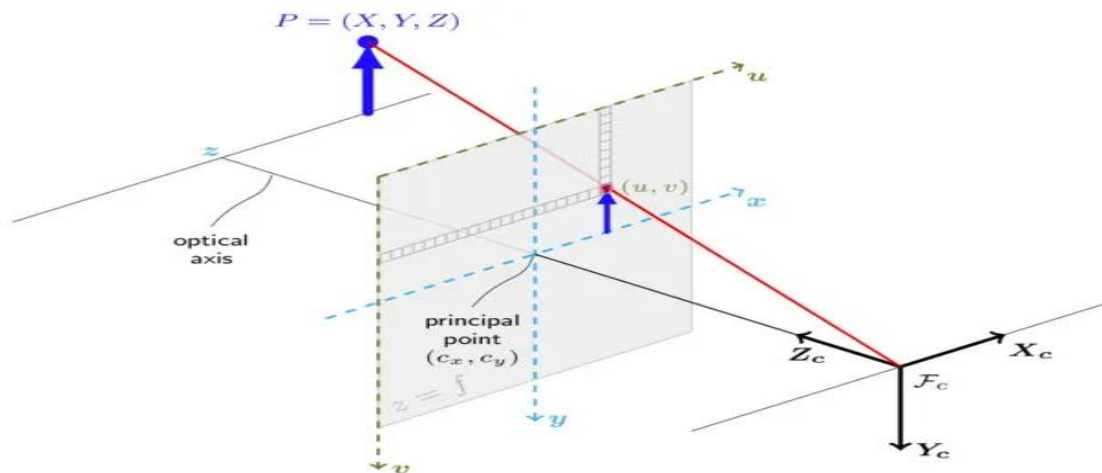


Figure 39: Visualising the Conversion of 2D image space into 3D world coordinates using pinhole camera model as state by the ZED2 camera documentation [22]

What are the formulas to get the 3D coordinates of an image pixel?

Given the 2D coordinates (u, v) of a pixel we can calculate the 3D coordinates (X, Y, Z) in the image frame (Z forward, X right, Y down) by using the following formulas:

```
Z = depth value of the pixel (u,v) from the depth map
X = (( u - c_x ) * Z) / (f_x)
Y = (( v - c_y ) * Z) / (f_y)
```

What are f_x , f_y , c_x , and c_y ?

They are the *intrinsic camera parameters* obtained with the camera calibration procedure, using the "pin hole" camera model:

- (f_x, f_y) : focal lengths in pixel units
- (c_x, c_y) : coordinates of the principal point

They are available runtime in the *CameraParameters* ([C++](#), [Python](#)) data structure of the ZED SDK.

Figure 40: Images from the documentation of ZED2 AI Stereo camera Documentation [22]

u,v is obtained from the depth data. X,Y,Z represents the world coordinates of the tracked object/person, from the python program above you can see that we used the same formula. This coordinates is how the robot is able to determine where the tracked person is.

Step 4: Conversion of 3D World Coordinates to Polar Coordinates

```
def cartesian_to_polar(x, y):
    # Calculate the radius (distance from camera origin to tracked person)
    radius = math.sqrt(x**2 + y**2)

    # Calculate the angle (in radians)
    angle = math.atan2(y, x)

    # Convert the angle from radians to degrees
    angle_degrees = math.degrees(angle)

    # Return the radius and angle in polar coordinates
    return radius, angle_degrees
```

Figure 41: Conversion of 3D World Coordinates to Polar Coordinates

We convert the cartesian world coordinates to polar coordinates (R and Theta) as they are used to decide **how far the tracked object is from the robot and in which direction** the robot should move.

- R: determines the distance from the camera to target.
- Theta: determines the angle to the target

Step 5: Movement of the robot

```
if subscriber.corr_array != []:

    my_pose.position.x = subscriber.corr_array[0]["x"]
    my_pose.position.y = subscriber.corr_array[0]["y"]
    my_pose.position.z = subscriber.corr_array[0]["z"]
    my_pose.orientation.w = 0
    my_pose.orientation.x = 0
    my_pose.orientation.y = 0
    my_pose.orientation.z = 1

    # rospy.loginfo(my_pose)
    pub.publish(my_pose)

    r, theta = cartesian_to_polar(subscriber.corr_array[0]["x"], subscriber.corr_array[0]["z"]) # in polar coordinate
    ### update velocity and publish ####
    print('r:',r)
    twist_msg = Twist()
    # Update linear Velocity
    if r > 1.1:
        twist_msg.linear.x = 0.08 * (r - 1.1) + 0.05
    elif r < 0.9:
        twist_msg.linear.x = 0.0

    twist_msg.angular.z = 0.008 * (theta - 90) # Update angular Velocity, oscillation exists

    pub_cmd.publish(twist_msg)
    print('succesfully published.')
```

Figure 42: Code showing how we move the robot

Dynamixel Workbench controller is a package that is responsible for moving the robot. With this, it allows the robot to base its path planning on where it is currently at and where the end-goal pose is at. By publishing a message containing the angular and linear velocity to the controller, the robot can move and adjust its movement accordingly to the tracked target. This depicts the beauty of ROS as we did not have to program everything from scratch.

Usage of polar coordinates (R and Theta) to update the linear and angular velocity respectively. The linear velocity is updated based on the distance (r) between the robot and the target. This nearer the robot is to you, the slower it gets. The further it is to you the faster it approaches you. This ensures the robot approaches or stops near the target as needed. Once velocity is updated, we publish command to dynamixel topic to move the robot in a certain direction at a certain velocity.

Why we chose YOLOv7 + ByteTrack over FairMOT to implement on the robot?

While FairMOT did outperform YOLOv7 + ByteTrack by a few percentage points in terms of tracking IDs in both MOTA and IDF1, we felt that it was not significant enough to justify using FairMOT. This is because FairMOT causes our Frames per second (FPS) to drop to around 8-9 fps in comparison to YOLOv7 + ByteTrack's averages around 23-24 Frames per second, this heavily goes against the whole premise of real-time object detection when what we want is not just highly accurate detection but also for it to work in real-time. Our camera which operates at 30 FPS has not gone down all the way to 8-9 FPS on average if we were to use FairMOT.

3.4 Lidar-CV Fusion

3.4.1 Introduction to EagerMOT

Considering the strengths and limitations of the above methods: LiDAR approach is strong in detecting and tracking in 3-dimensional space but restricted by the effective signal range; cameras deliver a detailed, high-density visual signal, aiding in the localization of objects even at a distance yet is limited to image domain. To maximize the strengths and minimize weaknesses for better object tracking, EagerMOT, an algorithm integrating the two methods is implemented to this project. Through utilising KITTI and NuScenes datasets, EagerMOT has achieved cutting-edge performance on Multi-Object Tracking (MOT) tasks [5].

3.4.2 EagerMOT

Figure 43 shows the overview of the EagerMOT Tracking Framework. At the start, object detections from different sensor modalities (Image-based detector coupled with a LiDAR/stereo-based 3D object detector) are obtained at the input. The detections then undergo fusion, and the objects detected will be configured to acquire both 3D and 2D parameters. It will be followed by a two-stage operation of data association which updates

the object tracking regardless of the detections being originated from either one or both sensor modalities. In the first stage, instances with 3D information (with or without 2D data) are paired with existing tracks. In the second stage, tracks that were previously not matched are associated with instances that are localized only in 2D.

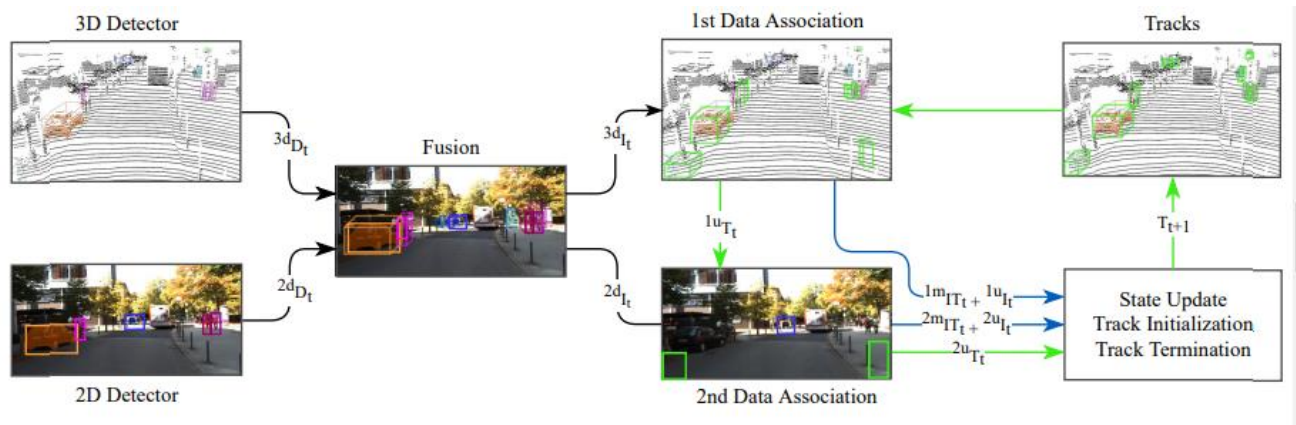


Figure 43: A high-level overview of EagerMOT Tracking Framework

The code for EagerMOT is split into 2 main parts, tracking and visualizing. The whole model is run on a pre-trained detection model, where we initially used KITTI models to track our bounding boxes, but realized that due to a lack of ID parameters, it will lead to issues in the future. This led us to pivot using a NuScenes detection model and Data. After tracking is done on the provided data, the visualization code can be run to see the results in a 3D program called Open3D, where the point cloud is combined with parameters gathered from the previous tracking algorithm to paint a 3D model of each scene.

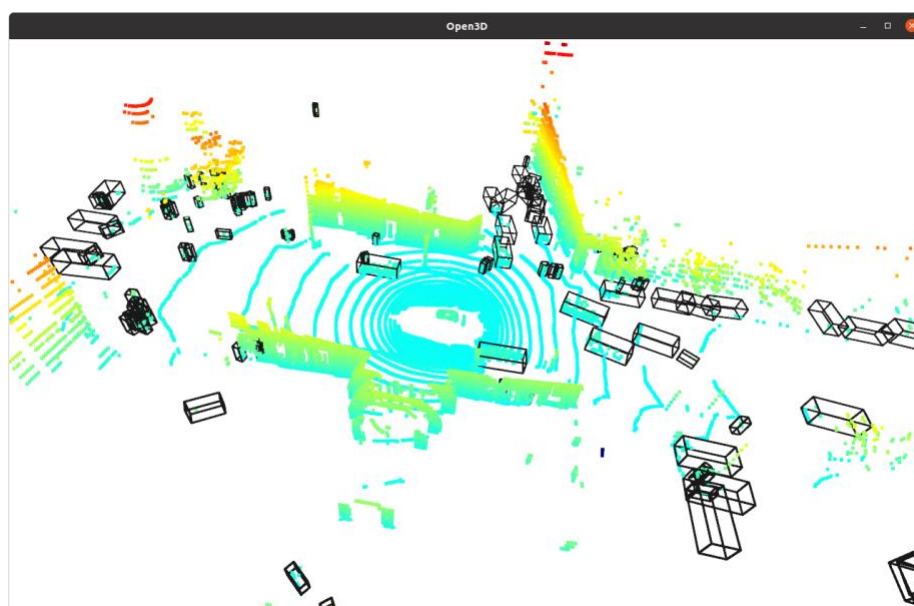


Figure 44: A basic overview of Open3D Visualization.

3.5 Data Collection and Labelling

We carried out data collection using two separate cameras over the course of three main sessions. Each session began with a different number of people at each camera—starting with 3, 4, and 5 participants, respectively—so we could get a better understanding of how well the setup would handle different crowd sizes. For each of these sessions, we organized three rounds where participants moved between the two cameras. In the first round, only one person from each camera switched spots. In the second round, two people moved between cameras, and in the final round, all participants were allowed to move freely between the cameras.

We used the ZED_Explore app to collect CV data and LivoxViewer2 to collect LiDAR data. Bounding boxes were automatically created by the YOLOv7 model. As the next step we labelled the bounding boxes by assigning a unique number to find the tracking ground truth.



Figure 45: Person Identification (ID) for Data Collection

Person ID	ID1	ID2	ID3	ID4	ID5
Sequence	3985	fd65	dbA5	FDC5	DBB5

Table 4a: Sequence ID of RFID tags for ID1-ID5

Person ID	ID6	ID77	ID88	ID99	ID1010
Sequence	FD45	3975	3995	FDD5	39A5

Table 4b: Sequence ID of RFID tags for ID6-ID10

We split the tests into 2 sets, Set A (stationary camera) and Set B (moving robot). 9 tests are conducted for each set and the conditions are as listed in Table 4a and 4b. Duration of all tests is 1.5 minutes each.

Test	Number of people	Number of persons changing
test1	6	2
test2	6	2
test3	6	random
test4	8	4
test5	8	4
test6	8	random
test7	10	6
test8	10	6
test9	10	random

Table 5a: Set A (Stationary camera) Test Set conditions

Test	Number of people	Number of people changing
test10	6	2
test11	6	2
test12	6	random
test13	8	4
test14	8	4
test15	8	random
test16	10	6
test17	10	6
test18	10	random

Table 5b: Set B (Moving) Test Set conditions

Finding errors was an extremely tedious task as we had to observe 36 videos, frame by frame, to find any discrepancies in the bounding boxes. Figure 45 shows the specific Person ID for data labelling and Table 4a and 4b shows the Sequence ID of RFID tags for each Person ID. We noted down the places of discrepancy so that those could be fixed before running the algorithm. The data labelling instructions followed are shown in Figure 46.

1. IDx only record non-conflict ID.

2. Conflict events only record conflict ID.

Example:

ID9: [71, 75]

ID6: [63, 69]

ID5: [65, 74]

ID2: [66]

ID10: [70, 73]

ID3: [60]

Conflict events: [[67, 1964, 9, 5], [aa, bbb, cccc, dddd],...]

Meaning: ID67, change frame in 1964, before 1964, ID 67 belongs to real ID9, after that belongs to ID5.

Figure 46: Data Labelling Instructions

4. Cost

Phase	Planned Cost	Actual Cost
Initiation	nil	nil
Planning	nil	nil
Execution	\$1000	\$999
Closing	nil	nil
Project Total Cost	\$1000	\$999

Table 6: Planned and Actual Costs

Name of Purchase	Price of Purchase
Asus RTX 4070 Super Prime OC 12GBDDR6X Graphics	\$999

Table 7: Purchased items

Our DIP group used the \$1000 dollars allocated to our group to purchase a powerful GPU to train and execute our models on. We decided to invest in a GPU because all other components for the project such as LiDAR sensors and the robot were previously available. The new GPU helped us reduce train and test times significantly. The expenditure (\$999) is slightly under the allocated budget of \$1,000.

5. Outcomes / Benefits

Throughout the 13 weeks, our project has reaped significant benefits and takeaways at the different phases.

5.1 Phase 1: Planning

The planning phase of our project took place in weeks 1 to 4. During the initial weeks, our group had to go through academic papers and online coding tutorials to gain insights on the gist of the project. These helped us to acquire essential background knowledge needed to work on the project. With the assistance of the Master's and PHD students, we were able to draft out the outline of our project, i.e. the tasks that needed to be completed by a certain date. The self-learning and reading, together with the schedule drafted by the Master's and PHD students helped to enhance the effectiveness and efficiency for our subsequent phases. We split into 3 sub-groups: LiDAR, CV and LiDAR-CV Fusion, to efficiently split the workload and delegate tasks.

5.2 Phase 2: Execution

During the execution phase, we tapped into our knowledge acquired in phase 1. The three sub-groups had to stick closely to the planned schedule to ensure that we were able to complete the project well on time. The execution phases included data training, collection, labelling and implementation of our project. This phase enabled us to apply what we had learnt in phase 1 of our project. We were able to understand the end-to-end process of how developing and training a model works, from data collection all the way to training and testing the model. There was a lot of trial-and-error involved, then eventually finding a solution.

5.3 Phase 3: Implementation

Finally, the robot was implemented. The robot was able to track a specific person or object continuously. We tested the robot by having a second person or object come into view. The robot did not change its target and continued to follow the first person. We also were able to match state-of-the-art results on all our fine-tuned model which was trained on the dataset which we collected and labelled ourselves.

6. Project Management Review

6.1 Project Initiation

While carrying out our project, we ensured that our goals were aligned with the SMART framework as highlighted in section 1.1 Introduction of our report.

6.2 Project Planning

During the formation of our group, we split ourselves into three groups and each of the teams carried out their roles diligently. With the help of our mentors, we had defined the tasks, deadlines, and responsibilities of each group.

6.3 Risk Management

During the execution of our project, we used a robot that was made up of four antennae, cameras, connecting wires as well as computers. As with all projects, we are bound to encounter risks that can delay or hinder our progress. One such risk is compromised software. To prevent exposure to such risks we took precautions, such as only using sources or downloading files from trusted sites such as GitHub. Carelessness in checking the credibility of the sources could lead to software bugs which may damage the computers. We also were able to only access our training server via VPN or when we are in school.

Additionally, equipment could become damaged from water spills and electrical faults. We took care not to use exposed or damaged cables for robot operation. Water bottles were kept out of the computers' way.

While such risks are plausible, they should be expected, and precautions must be taken to minimize the chances of them occurring.

6.4 Cost Management

Our group was given a budget of \$1000 to purchase equipment for our project. It was essential for us to ensure that the budget was able to cover the cost of all necessary equipment, which means sourcing for the most affordable yet effective equipment. As we had to follow a strict timeline to ensure that we meet our goals, the time taken to train our data had to be reduced. To do this, a more powerful GPU was needed as the existing one lacked power. The server, provided by the lab to use, has a few powerful GPUs, however there was some internal issue with the CPU. Thus, when we were trying to use the server to train, it had errors. After relentless debugging, we were able to run everything on the server, however it was very slow. Hence, it was decided that the budget would be to buy a more powerful GPU as we had 3 models to train.

7. Reflection

Throughout the 13 weeks, we have gained invaluable experiences. Prior to taking up this project, most of us had no prior knowledge in coding, data collection, model implementation/training or model evaluation. We were all united in our interest in machine learning and data analytics. We were unaware of how we could utilise camera and LiDAR sensors to work into our model. Initially, our project group reviewed research papers on the respective deep learning algorithms, which enabled us to gain insights of the methods and engineering knowledge that are essential. At the end of the project, we became more adept at machine learning and data analytics. We were also able to implement algorithms which gave comparable results to state-of-the-art models.

Inevitably, we had faced some challenges, such as errors in coding and issues with data labelling. Often, we had to spend much time on debugging the errors through assessing GitHub codes or sourcing solutions from CodeStack and StackOverFlow. This helped us learn that being meticulous and resourceful are important qualities for problem analysis. The data collection and labelling were new experiences as we usually just use data that has already been collected. This whole entire process helped us understand how the models really work and the entire process of machine learning.

To meet our objective, we managed to train 4 Models (YOLOv7 + Bytetrack, FairMOT, Centerpoint, EagerMOT) which integrates both camera and LiDAR data to enhance object tracking accuracy. Our three teams worked tirelessly and met up for weekly discussions, ensuring seamless communication. Our teamwork led to the birth of our solution, which was tested rigorously. For us to be on track, every member had to do their part in ensuring that the deadlines were met, highlighting the importance of individual contribution and teamwork.

While we eventually met our objectives, our team wishes to have explored further in this research. However, due to time constraints, we could not do more than what was targeted. Hence, our recommendations for future research include enhancing real-time processing capabilities of the system in complex environments, integrating more sensors such as thermal imaging to deal with extreme temperatures (given the rising temperatures due to global warming) and refining the fusion algorithm to better track objects in crowded spaces with unpredictable elements. By delving deeper into these aspects, we believe that autonomous robots would become more advanced and suitable to be used in public spaces.

8. Acknowledging/Declaring the Use of GAI

Please refer to NTU's Current Policy & Guidelines on the Use of Generative AI available in NTUlearn home page and the link:

[https://entuedu.sharepoint.com/sites/Student/dept/ctlp/SitePages/Exploring-the-Impact-of-Generative-Artificial-Intelligence-\(GAI\)-Tools-on-Education.aspx](https://entuedu.sharepoint.com/sites/Student/dept/ctlp/SitePages/Exploring-the-Impact-of-Generative-Artificial-Intelligence-(GAI)-Tools-on-Education.aspx)

- Complete the following declaration if applicable.
- Create a Paper Trail to document the input prompt, output obtained, and how you have used it

I, Nguyen Khanh Nam (student name), e220047@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Design & Innovation Project
2. Course Code: EE3180
3. Instructor: Assoc Prof Lin Zhiping
4. Title of Assignment/Project Submission: LiDAR and camera fusion for better robot tracking under complex environment Project Report

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

- i. Used GAI as permitted to assist in generating key ideas only. ☐
- ii. Used GAI as permitted to assist in generating a first text only. ☐
- And/or
- iii. Used GAI to refine syntax and grammar for correct language submission only. ☐
- Or
- iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project. ☒

I also declare that I have:

- a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
- b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
- c. Apart from the foregoing notices, the submission is wholly my own work.

Nam

2024

Student Name & Signature

Nguyen Khanh Nam

11 November

Date

References

- [1] T. Yin, X. Zhou, and Philipp Krähenbühl, "Center-based 3D Object Detection and Tracking," *arXiv (Cornell University)*, Jun. 2020, doi: <https://doi.org/10.48550/arxiv.2006.11275>.
- [2] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "PointPillars: Fast Encoders for Object Detection From Point Clouds," *IEEE Xplore*, Jun. 01, 2019. <https://ieeexplore.ieee.org/document/8954311>
- [3] Y. Yan, Y. Mao, and B. Li, "SECOND: Sparsely Embedded Convolutional Detection," *Sensors*, vol. 18, no. 10, p. 3337, Oct. 2018, doi: <https://doi.org/10.3390/s18103337>
- [4] X. Zhou, D. Wang, and Philipp Krähenbühl, "Objects as Points," *arXiv (Cornell University)*, Jan. 2019, doi: <https://doi.org/10.48550/arxiv.1904.07850>.
- [5] A. Kim, A. Osep, and L. Leal-Taixe, "EagerMOT: 3D Multi-Object Tracking via Sensor Fusion," 2021 IEEE International Conference on Robotics and Automation (ICRA), May 2021, doi: <https://doi.org/10.1109/icra48506.2021.9562072>.
- [6] Alex H. Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. Pointpillars: Fast encoders for object detection from point clouds. CVPR, 2019.
- [7] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. ICIP, 2016.
- [8] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022, April 7). *ByteTrack: Multi-object tracking by associating every detection box*. arXiv.org. <https://arxiv.org/abs/2110.06864>
- [9] Carranza-García, M., Torres-Mateo, J., Lara-Benítez, P., & García-Gutiérrez, J. (2020, December 29). *On the performance of one-stage and two-stage object detectors in autonomous vehicles using camera data*. MDPI. <https://www.mdpi.com/2072-4292/13/1/89>
- [10] Wang, C.-Y., Bochkovskiy, A., & Liao, H.-Y. M. (2022, July 6). *Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors*. arXiv.org. <https://arxiv.org/abs/2207.02696>
- [11] *Two-stage object detectors*. Ultralytics. (2024, October 24). <https://www.ultralytics.com/glossary/two-stage-object-detectors>

- [12] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016, May 9). *You only look once: Unified, real-time object detection*. arXiv.org. <https://arxiv.org/abs/1506.02640>
- [13] *A deep dive into non-maximum suppression (NMS)*. Built In. (n.d.). <https://builtin.com/machine-learning/non-maximum-suppression>
- [14] Rosebrock, A. (2024, November 10). *Intersection over union (IOU) for object detection*. PyImageSearch. <https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/>
- [15] O'Mahony, Sean Campbell, Anderson Carvalho, Suman Harapanahalli, N., Walsh, J., Riordan, D., Krpalkova, L., & Hernandez, G. V. (n.d.). *Deep learning vs. Traditional Computer Vision*. arxiv. <https://arxiv.org/pdf/1910.13796>
- [16] Redmon, J., & Farhadi, A. (2016, December 25). *Yolo9000: Better, faster, stronger*. arXiv.org. <https://arxiv.org/abs/1612.08242>
- [17] Ioffe, S., & Szegedy, C. (n.d.). *Batch normalization: Accelerating deep network training ...* arxiv. <https://arxiv.org/pdf/1502.03167>
- [18] Chun, L. Z., Zhi, J. Y., Jing, W., Zhang, C., & Dian1, L. (n.d.). *Architecture of yolov3. it consists of residual network, feature... | download scientific diagram*. researchgate. https://www.researchgate.net/figure/Architecture-of-YOLOv3-It-consists-of-Residual-Network-Feature-Pyramid-Networks-FPN_fig1_343694111
- [19] Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020, April 23). *Yolov4: Optimal Speed and accuracy of object detection*. arXiv.org. <https://arxiv.org/abs/2004.10934>
- [20] Mendez, M. (2024, August 25). *Understanding Object Tracking Metrics*. Miguel Mendez. <https://miguel-mendez-ai.com/2024/08/25/mot-tracking-metrics>
- [21] Fateshelled. (n.d.). *Fateshelled/bytetrack_ros: ByteTrack + Ros 2*. GitHub. https://github.com/fateshelled/bytetrack_ros?tab=readme-ov-file
- [22] *How can I convert 3D world coordinates to 2D image ...* (n.d.). <https://support.stereolabs.com/hc/en-us/articles/4554115218711-How-can-I-convert-3D-world-coordinates-to-2D-image-coordinates-and-viceversa>
- [23] Zhang, Y., Wang, C., Wang, X., Zeng, W., & Liu, W. (2021, October 19). *Fairmot: On the fairness of detection and re-identification in multiple object tracking*. arXiv.org. <https://arxiv.org/abs/2004.01888>

- [24] Le, B. (n.d.). *An introduction to bytetrack: Multi-object tracking by associating every detection box*. Datature. <https://www.datature.io/blog/introduction-to-bytetrack-multi-object-tracking-by-associating-every-detection-box>

Appendix A - Project Members Information

	Name	Project Contribution	Report Contribution
1	Annabella Choo	- LiDAR Sub-group Member - Literature Review - Data collection (was the sample object) - Data labelling - Assisted with setting up environment on test server for LiDAR - Assisted with the preparation of KITTI and NuScenes datasets - Wrote group blogs	- 1.1 Introduction - 2. Project summary (2.1 and part of 2.2) - 3.1.1 Introduction to CenterPoint - 3.1.2 CenterPoint Detection + Tracking models for NuScenes (a. Center Heatmap Head, c. Velocity head and tracking) - 3.1.3 CenterPoint Detection model for KITTI (a, b, c, d, e) [all the paragraphs] - 5. Outcomes/Benefits - 6. Project Management Review (6.1, 6.2, 6.3, part of 6.4) - 7. Reflection (entire) - Proofreading of report
2	Da Ruolin	- LiDAR-CV Fusion Sub-group Member - Literature Review - Setting up environment and preparing training dataset for fusion algorithm - Data labelling	- 3.4.1 Introduction to EagerMOT
3	Kyaw Tun Khine	- CV Sub-group Member - Literature Review - Training, testing and setting up of environment for both YOLOv7 + ByteTrack and FairMOT - Implementation of CV on Robot - Data collection and labelling - Wrote group blogs	- 1.1 Introduction - 1.2 Schedule - 3.3 Implementation of CV on Robot (entire thing) - 3.2 YOLOv7 + ByteTrack & FairMOT model (entire) - Proofreading of entire report

4	Lim Jing Xuan	- LiDAR Sub-group Member - Literature Review - Setting up environment on test server for LiDAR - Data labelling - Preparation of KITTI and NuScenes datasets - Wrote group blog	- Purpose and Project Objectives - 1.1 Introduction - 1.2 Schedule - 3.1.2 CenterPoint: b. Regression Heads - 3.1.2.2 Visualisation on NuScenes Detection Model - 3.5 Data Collection and Labelling - Project Management Review - Proofreading and formatting of entire report
5	Mindy Soh Min Qi	- LiDAR-CV Fusion Sub-Group Member - Data labelling - Literature review	- 3.4.2 EagerMOT
6	Nguyen Khanh Nam	- Group Leader - Training and setting up of environment for LiDAR Detection + Tracking Model (CenterPoint) - Data labelling - Assist in purchasing of the RTX 4070 Super graphics card	- 3.1.2.1 Testing CenterPoint NuScene models' results - 3.1.2.2 Visualization on NuScene Detection model - 3.1.2.1 Testing CenterPoint KITTI Detection - 3.1.3.2 Visualization on KITTI Detection model
7	Nitheesh Sundaram	- CV Sub-group Member - Assisted with few minor CV tasks. - Data Labelling - Sample Object for data collection.	- Gave verbal inputs related to CV.
8	Ravindran Ramalingam	- Treasurer - CV Sub-group Leader - Data Collection and Labelling - Lit Review	- 3.2 ByteTrack & FairMOT model - 3.5 Data Collection and Labelling - Cost (entire)

9	Zhang Yifan	- LiDAR-CV Fusion Sub-group Leader - LiDAR-CV Fusion Model Training and Visualizing code initializing and debugging.	- 3.4 Lidar-CV fusion, how does EAGERMOT code function.
---	-------------	---	---

Sub-Group Members List

LiDAR Sub-group:

Nguyen Khanh Nam (Project Manager)

Annabella Choo

Lim Jing Xuan

CV Sub-group:

Ravindran Ramalingam (Project Treasurer)

Kyaw Tun Khine

Nitheesh Sundaram

LiDAR-CV Fusion Sub-group:

Zhang Yifan (Sub-group Leader)

Da Ruolin

Mindy Soh Min Qi